

Mon, Wed, Fri → 3 classes

10  
labs

← Lab Assignment → Every week  
[Pytorch] Friday → Next Sunday

Lab → 25 m

Midsem → 25 m [MARCH 11]

Endsem → 50 m

## EE5175 IMAGE SIGNAL PROCESSING

→ Image formation  $\left\{ \begin{array}{l} \text{Pin-hole model} \\ \text{Real aperture [Camera]} \end{array} \right.$

Factors affecting image formation:

- ① 3D-geometry
- ② Motion
- ③ Illumination



- Noise (low-light)  $\rightarrow$  Poisson
- Blur (Out of focus, motion)  $\rightarrow$  Gaussian  $\downarrow$  No fixed model
- Weather [Fog, haze, rain, etc.]

- Resolution (Low line details, FPS)  $\rightarrow$  can't capture motion correctly with lower FPS
- Depth

5/11

→ 2 types of problem: 1) Forward

2) Inverse  $\rightarrow y = Ax + b \rightarrow \text{Finding } x$

Image Problems in  $\odot$  Space

→  $\odot$  Consumer cameras: (i) Image mosaicing: Panorama  $\rightarrow$  Combining set of 'overlapping' image to form a panorama

(ii) HDR imaging:

(iii) Low-light photography

(iv) Rolling shutter rectification: Row wise exposure delay causes skew & curvature distortions while imaging under camera motion

(vi) Motion deblurring

(vii) Dual-lens cameras: capture depth info

(viii) Stabilization: eg video captured in moving vehicle

Biometrics: Gait, Face recognition  
Iris, Fingerprint  $\rightarrow$  find pts  
 $\rightarrow$  detect angle  
 $\rightarrow$  form track  
island

→ Robotic navigation  $\rightarrow$  depth map

→ Structure from motion

→ Remote sensing: segmentation

→ Surveillance: Change detection  $\rightarrow$  detect dominant motion from all the persisting motions  
 $\rightarrow$  Rolling shutter

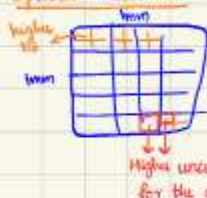
→ Medical image super-resolution

→ Digital heritage  $\rightarrow$  Reconstructing original structure from a damaged structure

→ Underwater deblurring

- **Pixel** - Picture element → Smallest unit of a digital image. Pixel matrix with diff. graylevel values forms a dig. image.
- **Digital image**: A 2-D signal (color, gray, binary).
- **Aspect ratio**: Picture width to picture length.
- **Brightness Resolution**: No. of bits used to represent graylevel values. e.g. 4 bits for 16 graylevels  
8 bits for 256 graylevels. } indpt of s/b
- **Spatial Resolution**: Ability to resolve details.
- **Temporal Resolution**:

**Spatial Resolution**: Depends on pixel density → No. of pixels packed in a given space.



- As we keep increasing the pixel density, shot noise comes into picture.
- Light entering the pixel is a random process.  
↓  
# of photons collected on pixel.

→ Let 'a' (Discrete RV) represent the no. of photons counted. The fluctuation in the no. of photons collected can be explained using a Poisson distribution. [Arrival of discrete RV → Poisson]

$$\Rightarrow a \sim \text{Poisson}(\lambda)$$

$$P(a=k) = \frac{e^{-\lambda} \lambda^k}{k!} \quad k=0, 1, 2, \dots$$

$$\begin{aligned} \rightarrow \text{Characteristic fn } \phi(s) &= E[e^{sk}] = \sum_{k=0}^{\infty} e^{sk} \frac{e^{-\lambda} \lambda^k}{k!} = e^{-\lambda} \sum_{k=0}^{\infty} \frac{\lambda^k e^{sk}}{k!} \\ &= e^{-\lambda} \sum_{k=0}^{\infty} \frac{(\lambda e^s)^k}{k!} \\ \phi(s) &= e^{-\lambda} e^{\lambda e^s} \end{aligned}$$

$$\rightarrow \text{First moment: } \left. \frac{d\phi(s)}{ds} \right|_{s=0} = e^{-\lambda} e^{\lambda e^s} [\lambda e^s] \Big|_{s=0} = \lambda_0 = m_1$$

$$\rightarrow \text{Second moment: } \left. \frac{d^2\phi(s)}{ds^2} \right|_{s=0} = e^{-\lambda} [e^{\lambda e^s} \lambda e^s + e^{\lambda e^s} \lambda e^s] = e^{-\lambda} [e^{\lambda} \lambda + e^{\lambda} \lambda^2] = \lambda + \lambda^2 = m_2$$

$$\rightarrow \text{Variance} = m_2 - m_1^2 = \lambda + \lambda^2 - \lambda^2 = \lambda \quad \rightarrow \text{Std. deviation} = \sqrt{\lambda}$$

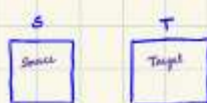
→ Assume on avg. 100 photons get collected → Uncertainty =  $100 \pm 10 \Rightarrow (10\%)$   
10 photons " " → " =  $10 \pm 3.33 \Rightarrow 33\%$  } ⇒ As pixel size ↓  
# of photons ↓  
s uncertainty ↑.

→ To generate low-light, scale down all values of  $\lambda$  by ( $<1$ ).



# Geometric transformation (2D) (In the image plane)

(i) Translation: Keeps angles & lengths intact



$$\begin{bmatrix} x_t \\ y_t \end{bmatrix} = \begin{bmatrix} x_s \\ y_s \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

Need not be integer

→ What kind of camera motion is required to get this kind of translation?? Specifically how the center shd move?

→ Also knowing  $t_x, t_y$  doesn't imply the motion of camera  $T_x, T_y$ ? Why? Depth comes into picture. (Camera is in 3D)

Soft zoom / Digital zoom v/s Optical zoom & Hardware

(ii) In-plane rotation

angles & lengths are preserved

$$\begin{bmatrix} x_t \\ y_t \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_s \\ y_s \end{bmatrix}$$

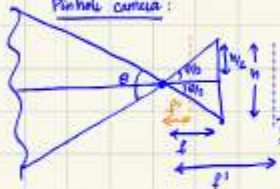
(iii) Uniform Scaling

$$\begin{bmatrix} x_t \\ y_t \end{bmatrix} = \begin{bmatrix} a & 0 \\ 0 & a \end{bmatrix} \begin{bmatrix} x_s \\ y_s \end{bmatrix}$$

→ Angles are intact  
Ratio of length are preserved

(iv) Optical zoom Hardware change

Pinhole camera:



$\theta$  → Field of view

$$\frac{\theta}{2} = \tan^{-1}\left(\frac{h}{f}\right) \Rightarrow \theta = 2 \tan^{-1}\left(\frac{h}{f}\right)$$

$f'$  → Field of view decreases → Telephoto length [More details]

$f''$  → increases → Wide angle photo

Soft zoom: (Used in cameras)

$$x_t = a x_s$$

$|a| > 1$  → Zoom in → Leaves holes in 'T'

$$y_t = a y_s$$

$|a| < 1$  → Zoom out

→ Taking source coordinates & mapping to target coordinates, then some of the pixels might not be visited which leaves holes in 'T'



Eg:  $a = 1.2 \rightarrow x_t = 1.2 x_s$

|       |     |     |     |     |     |     |
|-------|-----|-----|-----|-----|-----|-----|
| $x_s$ | 100 | 101 | 102 | 103 | 104 | 105 |
|-------|-----|-----|-----|-----|-----|-----|

|       |     |       |       |       |       |     |
|-------|-----|-------|-------|-------|-------|-----|
| $x_t$ | 120 | 121.2 | 122.4 | 123.6 | 124.8 | 126 |
|-------|-----|-------|-------|-------|-------|-----|

|         |     |     |     |     |     |     |
|---------|-----|-----|-----|-----|-----|-----|
| $[x_t]$ | 120 | 121 | 122 | 123 | 124 | 126 |
|---------|-----|-----|-----|-----|-----|-----|

|         |     |     |     |     |     |     |
|---------|-----|-----|-----|-----|-----|-----|
| $[x_t]$ | 120 | 122 | 123 | 124 | 125 | 126 |
|---------|-----|-----|-----|-----|-----|-----|

|           |     |     |     |     |     |     |
|-----------|-----|-----|-----|-----|-----|-----|
| Round off | 120 | 121 | 122 | 124 | 125 | 126 |
|-----------|-----|-----|-----|-----|-----|-----|

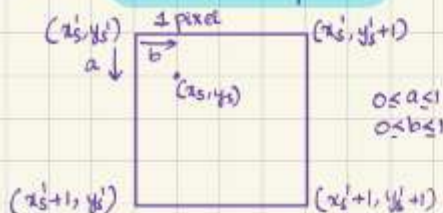
→ Lands inside a pixel → can use floor/ceil func<sup>n</sup> to get integer

126 → No 125  
126 → No 121  
126 → No 123

→ So source → target mapping is not done. Instead Target → Source mapping is done so as to visit every pixel in the target.

→ T → S Bilinear interpolation → Linear interpolation in 'x' & 'y' dir<sup>n</sup> →

Better image than S → T mapping



$$(x_s, y_s) = \left( \frac{x_t - b}{a}, \frac{y_t - b}{a} \right)$$

$$\Rightarrow x_s = \lfloor \frac{x_t - b}{a} \rfloor$$

$$y_s = \lfloor \frac{y_t - b}{a} \rfloor$$

$$b = y_s - y_s'$$

$$a = x_s - x_s'$$

Target intensity

$$I_t(x_t, y_t) = (1-a)(1-b) I_s(x_s', y_s') + (1-a)b I_s(x_s' + 1, y_s') + a(1-b) I_s(x_s + 1, y_s') + ab I_s(x_s + 1, y_s' + 1)$$

Similarity Transformation: Angles & ratios (of not necessarily parallel) lengths are preserved.

$$\begin{bmatrix} x_t \\ y_t \end{bmatrix} = S \cdot R \cdot \begin{bmatrix} x_s \\ y_s \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix} \rightarrow 4 \text{ unknowns}$$

Affine Transformation: Parallel lines remain parallel. Preserve ratios of lengths of  $\parallel^{\text{th}}$  segments.

$$\begin{bmatrix} x_t \\ y_t \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x_s \\ y_s \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix} \rightarrow 6 \text{ unknowns}$$

$\hookrightarrow$  x-shear  $\rightarrow$  saving ratios crossing from a pt.

2D

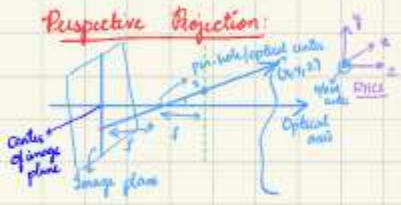
2-shear  $\begin{bmatrix} x_t \\ y_t \end{bmatrix} = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_s \\ y_s \end{bmatrix} \rightarrow$  if there is a y-shear.

Projective Transformation:

$L$  (3x3 matrix)

$$\begin{bmatrix} x_t \\ y_t \\ 1 \end{bmatrix} = \begin{bmatrix} \dots \\ \dots \\ \dots \end{bmatrix} \begin{bmatrix} x_s \\ y_s \\ 1 \end{bmatrix}$$

Perspective Projection:



$$\frac{x}{f} = \frac{x}{z} \Rightarrow x = \frac{fx}{z}$$

$$y = \frac{fy}{z}$$

can't write  $\frac{fy}{z}$  as matrix multiplication with  $x/y/z$  (need homogeneous coordinates)

Homogeneous coordinates

for a projection space

$$\begin{bmatrix} x_t \\ y_t \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_s \\ y_s \\ 1 \end{bmatrix} \Rightarrow \begin{bmatrix} x_t \\ y_t \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_s \\ y_s \\ 1 \end{bmatrix}$$

Submatrix of camera (K)

$\rightarrow$  In plane translation:

$$\begin{bmatrix} x_t \\ y_t \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_s \\ y_s \\ 1 \end{bmatrix}$$

$\rightarrow$  Rotation:

$$\begin{bmatrix} x_t \\ y_t \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_s \\ y_s \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x_t \\ y_t \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_s \\ y_s \\ 1 \end{bmatrix} \rightarrow \text{Affine}$$

Camera Intrinsic:  $(K) \rightarrow 3 \times 3$

$f \rightarrow$  focal length

$$\begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$K \rightarrow$  upper  $3 \times 3$  matrix

$\hookrightarrow$  with non-zero diagonal entries  
 $\downarrow$   
invertible

$\rightarrow O_x, O_y \rightarrow$  when optical axis does not pass through center of image plane

$\hookrightarrow$  2D translation

$$z x = f X + O_x z \Rightarrow x = \frac{f X}{z} + O_x$$

$$z y = f Y + O_y z \Rightarrow y = \frac{f Y}{z} + O_y$$

$$z = \bar{z}$$

$\rightarrow \frac{O_x}{z} \neq$  resolution  $\neq$  resolution  
 $\hookrightarrow$  take it into account

$\rightarrow s \rightarrow$  x-shear  $\rightarrow$  when  $x$  &  $y$  are not orthogonal



Zheng you Zhang

# Camera Extrinsic



How to relate one image with another when there is change in time/motion?

rotation abt z-axis  
by  $R_z$

$$R_z = \begin{bmatrix} \cos \theta_z & \sin \theta_z & 0 \\ -\sin \theta_z & \cos \theta_z & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

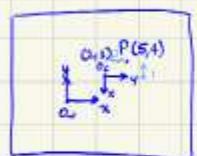
$$R_y = \begin{bmatrix} \cos \theta_y & 0 & -\sin \theta_y \\ 0 & 1 & 0 \\ \sin \theta_y & 0 & \cos \theta_y \end{bmatrix}$$

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta_x & \sin \theta_x \\ 0 & -\sin \theta_x & \cos \theta_x \end{bmatrix}$$

→ let  $R^{-1}$  be the matrix which translates  $O_w$  wrt  $O_c$ .

$$\underline{X_c} = R(\underline{X_w} - \underline{C}) = R\underline{X_w} - R\underline{C} = R\underline{X_w} + \underline{I} \Rightarrow \begin{bmatrix} \underline{X_c} \\ 1 \end{bmatrix} = \begin{bmatrix} R_{3 \times 3} & \underline{I} \\ \underline{0} & 1 \end{bmatrix} \begin{bmatrix} \underline{X_w} \\ 1 \end{bmatrix}$$

Ex:



Express 'P' wrt  $O_c$ :  $R = (-1, 0)$

$$\underline{X_w} = \begin{bmatrix} 5 \\ 4 \end{bmatrix} \quad \underline{C} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

$$\theta = 90^\circ \text{ CW} \\ R^{-1} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

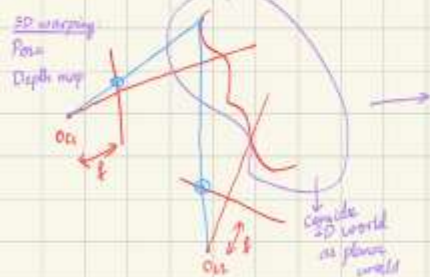
$$\underline{X_c} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \left( \begin{bmatrix} 5 \\ 4 \end{bmatrix} - \begin{bmatrix} 2 \\ 3 \end{bmatrix} \right) \Rightarrow R \cdot \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \\ = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 3 \end{bmatrix}$$

$$\rightarrow \begin{bmatrix} \underline{z} \\ \underline{y} \\ \underline{x} \end{bmatrix} = K_{3 \times 3} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \underbrace{\begin{bmatrix} R & \underline{I} \\ \underline{0} & 1 \end{bmatrix}}_{\text{Extrinsics}} \begin{bmatrix} \underline{X_w} \\ 1 \end{bmatrix}_{4 \times 1} \quad (\text{or}) \quad \begin{bmatrix} \underline{z} \\ \underline{y} \\ \underline{x} \end{bmatrix} = K \begin{bmatrix} R & \underline{I} \\ \underline{0} & 1 \end{bmatrix} \begin{bmatrix} \underline{X_w} \\ 1 \end{bmatrix}_{4 \times 1}$$

No rotation & translation  $\Rightarrow [R | \underline{I}] \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$

$$\underline{\tilde{z}} = P \underline{\tilde{X_w}} \rightarrow \text{Projection equation}$$

## Projective Transformation: (Homography)



$$\underline{\tilde{z}} = d \quad (w \times h)$$

Let coordinates of point 'P' be  $(x_1, y_1, z_1) \mapsto (1, 1, 1)$   
 $(x_2, y_2, z_2) \mapsto (2, 2, 2)$

Let the image coordinate of 'P' wrt  $O_c$  be  $(x_u, y_u)$

$$\underline{\tilde{z}} = \begin{bmatrix} \underline{z} \\ \underline{y} \\ \underline{x} \end{bmatrix} = k \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ 1 \end{bmatrix} = k \begin{bmatrix} x_1 \\ y_1 \\ z_1 \end{bmatrix} \\ \underline{\tilde{z}} = k \underline{\tilde{X_w}}$$

1st let the image coordinate of 'P' wrt  $O_c$  be  $(x_u, y_u)$

$$\begin{bmatrix} \underline{z} \\ \underline{y} \\ \underline{x} \end{bmatrix} = k \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ 1 \end{bmatrix} = k \begin{bmatrix} x_1 \\ y_1 \\ z_1 \end{bmatrix} = k \underline{\tilde{X_w}}$$

for diff. camera

$(x_2, y_2)$  wrt  $O_c$

$$\begin{bmatrix} \underline{z} \\ \underline{y} \\ \underline{x} \end{bmatrix} = k \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} R & \underline{I} \\ \underline{0} & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ 1 \end{bmatrix}$$

$$\underline{\tilde{X}_2} = R \underline{\tilde{X}_1} + \underline{I} \quad \text{where } \underline{I} = R \underline{C}$$





Homography computation  $H = k \left( R + \frac{T T^T}{\alpha} \right) K^{-1}$  → without knowing intrinsics, extrinsics

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

→ let the image coordinates b/w 2 views be related by a homography  $H$  as  $\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$

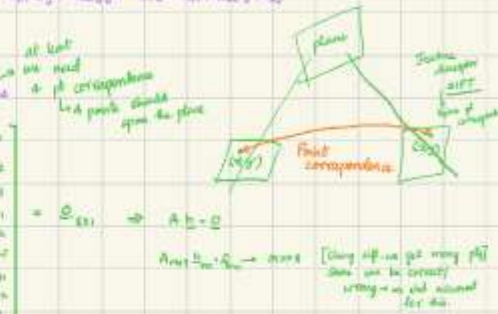
$$\begin{cases} \lambda x' = h_{11}x + h_{12}y + h_{13} \\ \lambda y' = h_{21}x + h_{22}y + h_{23} \\ \lambda = h_{31}x + h_{32}y + h_{33} \end{cases} \Rightarrow \begin{cases} x' \frac{\lambda x'}{\lambda} = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}} \Rightarrow h_{31}x x' + h_{32}y x' + h_{33}x = h_{11}x + h_{12}y + h_{13} \\ y' \frac{\lambda y'}{\lambda} = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}} \Rightarrow h_{31}x y' + h_{32}y y' + h_{33}y = h_{21}x + h_{22}y + h_{23} \end{cases}$$

→  $\|h\| = 2$

→  $H$  can only be found upto a scale factor. → 8 unknowns

$$\begin{cases} h_{11}x' + h_{12}y' + h_{13} - \lambda x = 0 \\ h_{21}x' + h_{22}y' + h_{23} - \lambda y = 0 \end{cases}$$

$$\begin{bmatrix} -x_1 & -y_1 & 1 & 0 & 0 & 0 & x_1 & y_1 & 1 \\ 0 & 0 & 0 & -x_2 & -y_2 & 1 & x_2 & y_2 & 1 \\ -x_3 & -y_3 & 1 & 0 & 0 & 0 & x_3 & y_3 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \\ h_{33} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \end{bmatrix}$$



$Ax = 0$  →  $Ax = 0$  [Using SVD we get wrong pt. since we can be correct! wrong → as not account for dist.]

→ Due to norm.

$$Ax \neq 0 \rightarrow \|Ax\| = 1$$

so we can minimize the norm of  $Ax$

$$\Rightarrow \min \|Ax\|^2 = (Ax)^T (Ax) = x^T A^T A x$$

→ Let  $x = y_1$  [So  $x$  gets multiplied with  $e_1^T$  which is smallest to minimize norm] [let col of  $e_1^T$  is  $\begin{bmatrix} 1 & 0 & \dots & 0 \end{bmatrix}$ ]

$$\Rightarrow \|Ax\|^2 = x^T V \Sigma^T \Sigma V^T x$$

$$= y_1^T V \begin{bmatrix} \sigma_1^2 & 0 & \dots & 0 \\ 0 & \sigma_2^2 & \dots & 0 \\ 0 & 0 & \dots & \sigma_r^2 \end{bmatrix} V^T \begin{bmatrix} y_1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = y_1^T y_1 e_1^T = e_1^T$$

using SVD  $A = U \Sigma V^T$   
 $U U^T = U^T U = I$   
 $V V^T = V^T V = I$   
 $\Sigma$  has singular values

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & 0 & \dots & 0 \\ 0 & \sigma_2 & 0 & \dots & 0 \\ 0 & 0 & \dots & \dots & \sigma_r \\ 0 & 0 & \dots & \dots & 0 \\ 0 & 0 & \dots & \dots & 0 \end{bmatrix}$$

$$\Rightarrow A^T A = V \Sigma^T U^T U \Sigma V^T$$

$$= V \Sigma^T \Sigma V^T$$

$$\Sigma^T \Sigma = \begin{bmatrix} \sigma_1^2 & 0 & \dots & 0 \\ 0 & \sigma_2^2 & \dots & 0 \\ 0 & 0 & \dots & \sigma_r^2 \\ 0 & 0 & \dots & 0 \end{bmatrix}$$

$\sigma_1 \rightarrow$  largest  
 $\sigma_r \rightarrow$  smallest } ordered singular values

→  $x_1 = y_1$  to minimize  $\|Ax\|^2$

using known  $K_1, R, T, \Sigma$

→ To compute  $H_1 (A \neq 0)$  perform SVD on matrix  $A$  and then choose the vector of  $V$  with smallest singular value



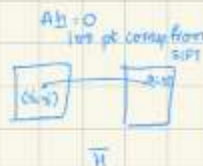
11/21

RANSAC (1981) Random Sample Consensus

→ Used to fit parameters of a model in the presence of outliers.

→ RANSAC uses min. no. of parameters needed to explain the model.

- Pick any  $\frac{n}{2}$  point correspondences and solve for  $H$ .  
→ min. no. of parameters
- Use this to calculate  $\begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} = H \begin{bmatrix} x_i' \\ y_i' \\ 1 \end{bmatrix}$  for the remaining points.



- Get the image coordinates as  $x_i'' = \frac{x_i'}{z_i'}$ ,  $y_i'' = \frac{y_i'}{z_i'}$

- Find  $\sqrt{(x_i'' - x_i'')^2 + (y_i'' - y_i'')^2} = \epsilon$

 $x_i'$  - Given feature correspondences as obtained from SIFT (vng).

- If  $\epsilon < \text{threshold}$ , then increment the consensus set.
- If the total no. of consensus pts. than a certain threshold, then stop and go to the final step. Else, continue for a fixed no. of iterations and choose 'H' with max consensus.
- Use all the inliers and compute 'H'.

Rotational homography:  $T \cdot O \Rightarrow H = K R K^{-1} \rightarrow$  depth does n't come into picture.Motion blur

→ All of the images gets averaged  
(due to small motion + diff view of scene)

$$g(x,y) = \frac{1}{T} \int_0^T f(H(x,y,t)) dt$$

$f \rightarrow$  actual intensity of the view  
Homography at time 't'

To remove 'time' component, consider some no. of pts which the centre could be and calculate the weighted sum depending on the fraction of time.

$$\Rightarrow g(x,y) = \int_{\lambda \in P} h(\lambda) f(H_\lambda(x,y)) d\lambda$$

$\lambda \in P$  set of all poses

$$\Rightarrow g(i,j) = \sum_{\lambda \in P} h(\lambda) f(H_\lambda(i,j)) \quad \text{where } \sum_{\lambda \in P} h(\lambda) = 1$$

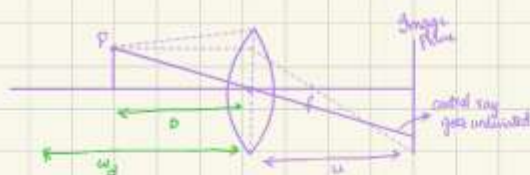
$h(\lambda) \geq 0$   
↓  
fraction of exposure.

→ Big picture: Able to make a video out of the blurred image.

# 13.11 Real Aperture (RA) cameras:

Why lens instead of pinhole?

- Gather more light within the same time duration  $\rightarrow$  Reduction in exposure time (very imp for dynamic scene)
- Can't increase aperture hole <sup>size</sup> (multiplying of rays causes blur)



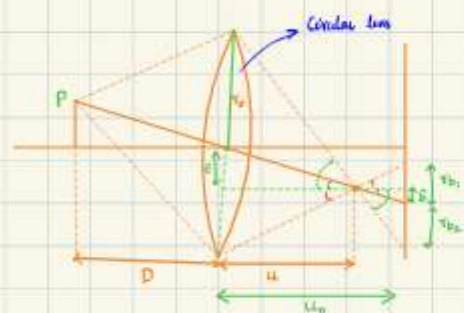
$\rightarrow$  2D imaging s/n is LSI (Linear Shift Invariant)

$\rightarrow$  Working distance -  $u_0$

$\hookrightarrow$  Object at this dist, the image is sharp

$\rightarrow$  Depth of field - depends on the type of lens (m to  $\mu$ m) <sup>mobile camera</sup>  
 $\hookrightarrow$  dist b/w 2 objects for both of them to be sharp

$\rightarrow$  Lens law:  $\frac{1}{f} = \frac{1}{u} + \frac{1}{v}$



\* ST  $T_{D1} = T_{D2} = T = T_0 \left( \frac{u_0 - u}{u} \right) = T_0 u_0 \left( \frac{1}{u} - \frac{1}{u_0} \right)$

$\frac{E}{u} = \frac{S}{u_0 - u} \quad \text{--- (1)}$

$\frac{T_0 - E}{u} = \frac{T_{D1} - S}{u_0 - u} \Rightarrow T_0 = \frac{S u}{u_0 - u} = T_0 \left( \frac{u}{u_0 - u} \right) = \frac{S u}{u_0 - u}$

$\frac{T_0 + E}{u} = \frac{T_{D2} + S}{u_0 - u} = \frac{T_0 + S}{u_0 - u} = \frac{u}{u_0 - u} \Rightarrow T_0 + S = T_0 \frac{u}{u_0 - u} + S \frac{u}{u_0 - u}$

$\Rightarrow T_{D2} = T_0 \left( \frac{u_0 - u}{u} \right)$

$\rightarrow \frac{1}{f} = \frac{1}{u} + \frac{1}{v}$   
 $\frac{1}{f} = \frac{1}{u_0} + \frac{1}{v_0}$   
 $\frac{1}{u} = \frac{1}{f} - \frac{1}{v} = \frac{1}{u_0} + \frac{1}{v_0} - \frac{1}{v}$

$\Rightarrow \frac{1}{u} = \frac{1}{f} - \frac{1}{v} = \frac{1}{u_0} + \frac{1}{v_0} - \frac{1}{v}$

$\Rightarrow S_0 = T_0 u_0 \left( \frac{1}{f} - \frac{1}{v} - \frac{1}{u_0} \right) = T_0 u_0 \left( \frac{1}{u_0} - \frac{1}{v} \right) = S_0$

Blue radius  $r_0$   
 $\begin{cases} S = 0 & \text{for } D < u_0 \\ S > 0 & \text{for } D > u_0 \rightarrow \text{the rays meet before the image plane} \\ S < 0 & \text{for } D < u_0 \rightarrow \text{the rays meet behind the image plane} \end{cases}$

$\rightarrow$  Is it linear? space invariant?  $\rightarrow$  Homogeneity } superposition  
 Additivity



what is i/p??

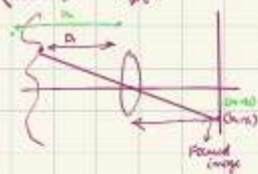
Linearity Assume a point light source at depth  $D$  that results in focused image intensity on the image plane  $(x_i, y_i)$ . This is as seen through a pin-hole. If we replace the pin-hole with a lens (all else remaining the same), then

$S_{D1}(x-x_i, y-y_i) \rightarrow S_1(x-x_i, y-y_i)$   
 $\frac{1}{f} = \frac{1}{u} + \frac{1}{v}$   
 (Riemann delta)  $\downarrow$   $\downarrow$   $\downarrow$   
 focus is not plane  $\downarrow$   $\downarrow$   $\downarrow$   
 if  $D = u_0$ ;  $S_1(x-x_i, y-y_i) = S(x-x_i, y-y_i)$

else it will form a blurred circle

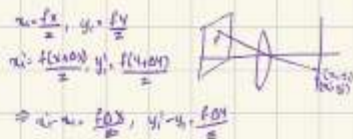
$S_{D1}(x-x_i, y-y_i) \rightarrow S_1(x-x_i, y-y_i) \rightarrow$  Homogeneity holds

$S_{D2}(x-x_i, y-y_i) \rightarrow S_2(x-x_i, y-y_i) \rightarrow$  Timeing of  $D$  is only have  $S_2$



→ When both sources are 'on' at the same time,  $S_0(x-y, z) + S_0(x+y, z) \rightarrow S_0(x-y, z) + S_0(x+y, z) \Rightarrow$  Additivity holds

→ Space invariant:  $S(x, y) \rightarrow g(x, y)$   
 $S(x-x_0, y-y_0) \rightarrow g(x-x_0, y-y_0)$   
 Suppose we translated by  $x_0, y_0$  then for a pt.  $P(x, z)$  the image coordinates before & after translation  
 $g(x, y, z)$   
 $x_1 = \frac{f}{z} x, y_1 = \frac{f}{z} y$   
 $x_2 = \frac{f}{z} (x+x_0), y_2 = \frac{f}{z} (y+y_0)$   
 $\Rightarrow x_2 - x_1 = \frac{f}{z} x_0, y_2 - y_1 = \frac{f}{z} y_0$



→  $S(x-x_0, y-y_0) \rightarrow g(x-x_0, y-y_0) \rightarrow y_{es} \Rightarrow$  space invariant  
 $\frac{f \cdot x_0}{z}, \frac{f \cdot y_0}{z}$

→ No LST =  $S(m, n) \rightarrow h(m, n; 0, 0)$   
 $S(m-n, n) \rightarrow h(m, n; m, n)$

LST  $\Rightarrow f(m, n) \xrightarrow{\text{LST}} \sum_m \sum_n f(m', n') \delta(m-m', n-n')$   
 $= \sum_m \sum_n f(m', n') \delta(m-m', n-n')$   
 $\xrightarrow{\text{LST}} \sum_m \sum_n f(m', n') \delta(m-m', n-n')$   
 $\rightarrow$  2D convolution

LSI

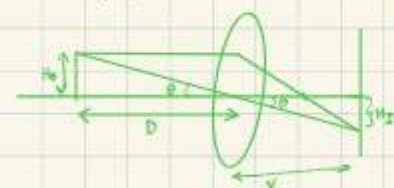
→  $h(m, n) \rightarrow$  PSF (Point spread function)  
 ↳ Not a filter applied but shows how intensity spreads when exposed to lens

→  $h(m, n) \rightarrow$  low pass filter  
 $h(m, n) \geq 0$   
 $\sum_m \sum_n h(m, n) = 1$

Depth of field: The range of depths at which the image comes out sharp

→ Pin-hole camera has = depth of field  
 → DSLR → large depth of field (compared to phone camera)  
 → Also changes with the dist. b/w the scene & lens

Lens magnification



Lens magnification =  $\frac{H_i}{H_o}$   
 $\tan \theta = \frac{H_o}{D}, \frac{H_o}{D}$   
 $\frac{1}{f} = \frac{1}{D} + \frac{1}{v}$

$$\frac{1}{f} = \frac{1}{D} + \frac{1}{v}$$

$$\Rightarrow \frac{H_i}{H_o} \cdot \frac{v}{D} = \frac{f}{D-f} = M$$

→  $M = \frac{f}{D-f} \Rightarrow D-M-f = f$   
 $D-M = f(1+M) \Rightarrow \frac{D}{f} = 1 + \frac{M}{1+M} = 1 + \frac{1}{M}$

→  $\text{DOF} = 2 \left( 1 + \frac{1}{M} \right)^2 f_{\text{num}} \cdot c$   
 ↳ microscope has high 'M' so smaller DOF

→ Pinhole smaller aperture  
 related to aperture  
 smaller aperture  $\Rightarrow$  larger  $f_{\text{num}}$   
 ↳ eg. 0.005mm  
 ↳ circle of confusion  
 depending on application  
 we need to fix 'c'  
 $\uparrow c \Rightarrow$  more & more  
 blurring can be  
 accepted as sharp  
 (defined by us)  
 (Willing to tolerate more blur)



LSI (2D convolution):

→ 1D convolution:

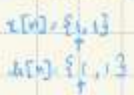
$$y[n] = \sum_{m=0}^2 h[m] x[n-m]$$

$$y[n] = h[0]x[n] + h[1]x[n-1]$$

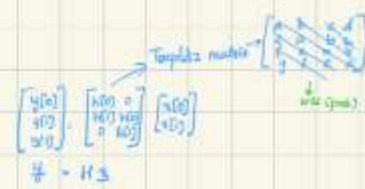
$$n=0, 1, 2$$

$$y[0] = h[0]x[0]$$

$$y[1] = h[0]x[1] + h[1]x[0]$$



$$y[2] = h[1]x[1]$$



→ Linear convolution through circular convolution (Zero padding  $h[n], x[n] \rightarrow$  get length  $= L-1$ )

$$y[n] = \sum_{m=0}^2 h[m] x[n-m]$$

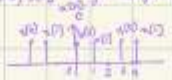
$$y = Hx$$

circulant matrix

$$\begin{bmatrix} y[0] \\ y[1] \\ y[2] \end{bmatrix} = \begin{bmatrix} h[0] & h[1] & 0 \\ h[1] & h[0] & 0 \\ 0 & h[1] & h[0] \end{bmatrix} \begin{bmatrix} x[0] \\ x[1] \\ x[2] \end{bmatrix}$$

Normal  $AA^T = A^T A$

Signal matrix  
Right circulant structure  
Toeplitz



2D

→ Gaussian and impulse train have the same shape in  $x(t)$  &  $X(\omega)$

2D-convolution:

$h, f$  exist only if  $0 \leq m, n \leq 2$



$$g = f * h$$

$$g(m,n) = \sum_{m'=0}^2 \sum_{n'=0}^2 f(m',n') h(m-m', n-n')$$

$$g(m,n) = f(0,0)h(m,n) + f(0,1)h(m,n-1) + f(0,2)h(m,n-2) +$$

$$f(1,0)h(m-1,n) + f(1,1)h(m-1,n-1) + f(1,2)h(m-1,n-2) +$$

$$f(2,0)h(m-2,n) + f(2,1)h(m-2,n-1) + f(2,2)h(m-2,n-2)$$

→ We do a lexicographical ordering for bringing out structure

$$\begin{bmatrix} g(0,0) \\ g(0,1) \\ g(0,2) \\ g(1,0) \\ g(1,1) \\ g(1,2) \\ g(2,0) \\ g(2,1) \\ g(2,2) \end{bmatrix} = \begin{bmatrix} h_{00} & 0 & h_{11} & 0 & 0 & 0 & h_{10} & 0 & h_{20} \\ h_{11} & h_{00} & 0 & 0 & 0 & 0 & h_{21} & h_{10} & 0 \\ 0 & h_{11} & h_{00} & 0 & 0 & 0 & 0 & h_{21} & h_{10} \\ h_{10} & 0 & h_{11} & h_{00} & 0 & h_{21} & 0 & 0 & 0 \\ h_{21} & h_{10} & 0 & h_{11} & h_{00} & 0 & 0 & 0 & 0 \\ 0 & h_{21} & h_{10} & 0 & h_{11} & h_{00} & 0 & 0 & 0 \\ 0 & 0 & 0 & h_{10} & 0 & h_{11} & h_{00} & 0 & h_{21} \\ 0 & 0 & 0 & h_{21} & h_{10} & 0 & h_{11} & h_{00} & 0 \\ 0 & 0 & 0 & 0 & h_{21} & h_{10} & 0 & h_{11} & h_{00} \end{bmatrix} \begin{bmatrix} f(0,0) \\ f(0,1) \\ f(0,2) \\ f(1,0) \\ f(1,1) \\ f(1,2) \\ f(2,0) \\ f(2,1) \\ f(2,2) \end{bmatrix}$$

→ The submatrix are individually circulant also circulant blockwise

$$g = Hf$$

↳ doubly block circulant matrix

→ DFT matrix has a special affinity towards doubly circulant matrix

$$H^T H = H H^T \rightarrow \text{Normal}$$

# Blue models - PSF models

(i) Fill-box model (ray geometry) → cont. domain

$$h(x,y) = \begin{cases} \frac{1}{\pi r_b^2} & 0 \leq x^2+y^2 \leq r_b^2 \\ 0 & \text{else} \end{cases}$$

since  $\int \int h(x,y) dx dy = 1$

→ Rays entering is equally distributed inside the circle of radius  $r_b$

→ Calibration constant

$$r_b = r_0 \lambda_0 \left( \frac{1}{\lambda_b} - \frac{1}{\lambda_0} \right)$$

$r_b$  - mm  
 $r_0$  - calibration const  
 $\lambda_0, \lambda_b$  → pixel/mm

## (ii) Gaussian blue model:

Lens suffers

- diffraction effect
- spherical aberration
- chromatic aberration
- vignetting

} 2D ⇒ CLT turns out to be Gaussian

Kernel Filter PSF

$$h(x,y) = \frac{1}{2\pi\sigma^2} e^{-\left(\frac{x^2+y^2}{2\sigma^2}\right)}$$

$$h(w,v) = e^{-\left(\frac{w^2+v^2}{2}\right)} \text{ where } \sigma = P \cdot r_b$$

→ Size of 'h' is crucial i.e. 11x11 causes severe blurring hence we do discrete approx of 'h'

size = ceil(6σ+1) next odd value

→ if  $\sigma = 1$ , size = ceil(6+1) = 7

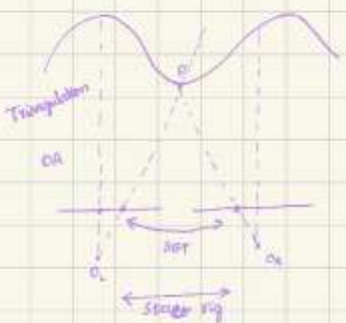
→ if  $\sigma = 2$ , size = ceil(7\*2+1) = 9

3D (Shape from x)  
 Depth → cue

→ What helps to decide depth:

- Focus/blur
- Shading
- Defocus → Further from lens less focus

## Pure Stereo (3d plane translation of the camera along the x-dir)



Let the camera intrinsics be  $K_1$  or  $K_2$

$$K_1 = \begin{bmatrix} f_{x1} & s_1 & o_{x1} \\ 0 & f_{y1} & o_{y1} \\ 0 & 0 & 1 \end{bmatrix} \quad K_2 = \begin{bmatrix} f_{x2} & s_2 & o_{x2} \\ 0 & f_{y2} & o_{y2} \\ 0 & 0 & 1 \end{bmatrix}$$

Let the co-ord of P be  $(x_0, y_0, z_0) = X_0$  wrt  $O_1$   
 $(x_1, y_1, z_1) = X_1$  wrt  $O_2$

$$X_1 = R X_0 + T$$

No rotation ⇒  $R = I$

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\Rightarrow \begin{bmatrix} f_{x1} & s_1 \\ f_{y1} & s_2 \\ z_1 & z_0 \end{bmatrix} = K_1 \begin{bmatrix} x_0 \\ y_0 \\ z_0 \end{bmatrix} \quad \begin{bmatrix} f_{x2} & s_2 \\ f_{y2} & s_1 \\ z_2 & z_0 \end{bmatrix} = K_2 \begin{bmatrix} x_0 \\ y_0 \\ z_0 \end{bmatrix}$$

$$= \frac{f_{x2}}{f_{y2}} = K_2 \begin{bmatrix} x_0 \\ y_0 \\ z_0 \end{bmatrix}$$

$$\Rightarrow x_R = \frac{f_x}{f_y} \cdot \frac{f_y x_L + f_z y_L + O_{Lx}}{z} = f_x \frac{x_L}{z} + f_x \frac{y_L}{z} + O_{Lx}$$

$$y_R = \frac{f_y y_L}{f_x} = f_y \frac{x_L}{z} + f_y \frac{y_L}{z} + O_{Ly}$$

$$\text{let } y_L' = \frac{y_L - O_{Ly}}{z} = \frac{y_L}{z}$$

$$x_L' = \frac{x_L - f_x \frac{y_L - O_{Ly}}{z} - O_{Lx}}{z} = \frac{x_L}{z}$$

→  $x_L', y_L', z_L'$  are normalized coordinates

→ If the cameras are different, normalized coordinates take acct of the intrinsics

$$\rightarrow x_L = \frac{f_L x_L'}{z_L} = \frac{f_{Lx} x_L' + f_{Lz} z_L' + O_{Lx}}{z_L} = \frac{f_{Lx} x_L'}{z_L} + O_{Lx}$$

$$y_L = \frac{f_{Ly} y_L'}{z_L} = \frac{f_{Ly} y_L'}{z_L} + O_{Ly}$$

$$\text{let } y_L' = \frac{y_L - O_{Ly}}{f_{Ly}} = \frac{y_L}{z_L} = y_L'$$

$$x_L' = \frac{x_L - f_{Lx} \frac{y_L - O_{Ly}}{f_{Ly}} - O_{Lx}}{f_{Lx}} = \frac{x_L + T_x}{z}$$

$$x_L' = x_R' + \frac{T_x}{z} \Rightarrow x_L' - x_R' = \frac{T_x}{z} \Rightarrow \boxed{z = \frac{T_x}{x_L' - x_R'}} \rightarrow \text{disparity}$$

Baseline  
i.e. dist. bet. centers of the cameras

$$\left( \frac{x_L - f_{Lx} \frac{y_L - O_{Ly}}{f_{Ly}} - O_{Lx}}{f_{Lx}} \right) - \left( \frac{x_R - f_{Rx} \frac{y_R - O_{Ry}}{f_{Ry}} - O_{Rx}}{f_{Rx}} \right) = \frac{T_x}{z}$$

→ Higher the disparity, closer the 'z'

i.e. object at further dist. has displacement in image

25/12

Special case: Identical cameras → same intrinsics

$$K = \begin{bmatrix} f_x & 0 & O_{Lx} \\ 0 & f_y & O_{Ly} \\ 0 & 0 & 1 \end{bmatrix}$$

$$\Rightarrow x_L' = \frac{x_L - f_x \frac{y_L - O_{Ly}}{f_y} - O_{Lx}}{f_x} = \frac{x_R + T_x}{z}$$

$$y_L' = \frac{y_L - O_{Ly}}{f_y} = \frac{y_R}{z} = y_R'$$

$$x_R' = \frac{x_R - O_{Rx}}{f_x} = \frac{x_R}{z}$$

$$y_R' = \frac{y_R - O_{Ry}}{f_y} = \frac{y_R}{z}$$

$$x_L' - x_R' = \frac{x_L - x_R}{f_x} = \frac{T_x}{z}$$

$$\Rightarrow \boxed{z = \frac{f_x T_x}{x_L - x_R}} \rightarrow \text{show does not come into picture}$$

→ RA camera (defocus/focus) → cue for depth

(Depth from defocus) (shape from focus) → focal stack

↓  
keyframing



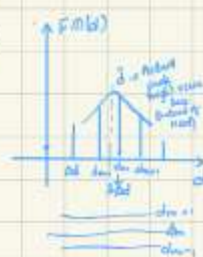
As we keep moving the stage up towards the plane of focus

→ we can see diff. blur for each point in each translation

→ When a pt reaches plane of focus, image is sharp

→ we can take a focal stack (say 80 frames)

→  $\vec{d}$  - distance vector moved by a pt to get focused.  
diff. pts have diff  $\vec{d}$   
if empty have  $\vec{d} = 0$   
 $\vec{d} \cdot \vec{s} \cdot \vec{d} \rightarrow$  this pt is further





→ Telecentric lens - specialized imaged lens to avoid parallax error, offering const magnification regardless of objects dist from lens

## Gaussian interpolation:

$$\begin{aligned} F_m &= F_p e^{-\frac{(dm-\bar{d})^2}{2\sigma^2}} \\ F_{m+1} &= F_p e^{-\frac{(dm+1-\bar{d})^2}{2\sigma^2}} \\ F_{m-1} &= F_p e^{-\frac{(dm-1-\bar{d})^2}{2\sigma^2}} \end{aligned}$$

3 unknowns  
 $F_p$  - peak value  
 $\sigma$  - spread of Gaussian  
 $\bar{d}$

$$\begin{aligned} \Rightarrow \ln F_m &= \ln F_p - \frac{(dm-\bar{d})^2}{2\sigma^2} \\ \ln F_{m+1} &= \ln F_p - \frac{(dm+1-\bar{d})^2}{2\sigma^2} \\ \ln F_{m-1} &= \ln F_p - \frac{(dm-1-\bar{d})^2}{2\sigma^2} \end{aligned} \quad \left. \begin{array}{l} \\ \\ \end{array} \right\} \text{solving}$$

$$\bar{d} = \frac{(\ln F_m - \ln F_{m-1})(dm+1 - dm^2) - (\ln F_m - \ln F_{m+1})(dm-1 - dm^2)}{2\Delta d (2\ln F_m - \ln F_{m+1} - \ln F_{m-1})}$$

$$\bar{d} = m + \frac{1}{2} \frac{\ln F_{m+1} - \ln F_{m-1}}{2\ln F_m - \ln F_{m+1} - \ln F_{m-1}}$$

$$F_p = \frac{F_m}{e^{-\frac{(dm-\bar{d})^2}{2\sigma^2}}}$$

$$\begin{aligned} dm+1 &= (m+1)\Delta d \\ dm-1 &= (m-1)\Delta d \\ dm &= m\Delta d \end{aligned}$$

$$\sim m + \frac{1}{2} \frac{F_{m+1} - F_{m-1}}{2F_m - F_{m+1} - F_{m-1}}$$

$$\sigma^2 = \frac{(dm+1-\bar{d})^2 - (dm-\bar{d})^2}{2(\ln F_m - \ln F_{m-1})} \rightarrow \begin{array}{l} F_p \text{ should be high} \\ \sigma^2 \text{ should be low} \end{array} \left\{ \begin{array}{l} \text{To accept} \\ \text{low measure} \end{array} \right.$$

→ How to sense if a pt is in focus??

→ Sum of focus operator should be zero.

## Focus operators:

$$\sum f_i = 0$$

(i) Point-wise: Gray scale intensity → check all pts in stack & see where max intensity occurs.

↳ NOT used much (slow but gets affected by noise).

(ii) Variance: Variance around a pt (avg)  $\frac{1}{n} \sum$

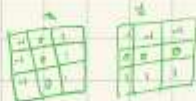
$$\text{Var}(x) = \frac{1}{M} \sum_{z \in \text{ROI}} (f(z) - \mu)^2$$

$\downarrow$  Avg. over all pts  
 $\downarrow$  window of ROI area  
 min square around (x,y)

(iii) Energy of the gradient  $\square$

$$\text{EOG}(z) = \sum (f_x^2(z) + f_y^2(z))$$

$\downarrow$  gradient along x  
 $\downarrow$  gradient along y



(iv) Laplacian operator (based on Sobel mask)  $\square$  → used for edge detection

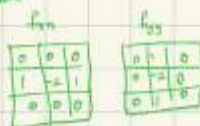
$$\text{Lap}(z) = \sum_{z \in \text{ROI}} (f_{xx}(z) + f_{yy}(z))$$

$$\nabla_x = \begin{bmatrix} -1 & 0 & 1 \\ 1 & 0 & 1 \\ -1 & 1 & 1 \end{bmatrix}$$

$$\nabla_y = \begin{bmatrix} 1 & -2 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

(v) Sum-modified Laplacian  $\nabla^2 = f_{xx} + f_{yy} \rightarrow$  used

$$\text{ML}(z) = \sum_{z \in \text{ROI}} (f_{xx}(z) + f_{yy}(z))$$



Second derivative of image along x & y dir

$$\text{SML}(f(x,y)) = \sum_{i=0}^{255} \sum_{j=0}^{255} \text{ML}(f(i,j))$$

2.1.1

(iv) LQG (Laplacian of Gaussian): BPF

$$LQG(x) = \sum_{z \in U(x)} \nabla_z^2 (f(z))$$

$$\nabla_z^2 = f_{xx} + f_{yy}$$

(v) Energy of Laplacian:  $EOL(x) = \sum_{z \in U(x)} (f_{xx}(z) + f_{yy}(z))^2$

Image Transforms (Unitary)

← Data input → DFT, DCT, DST  
← Data dependent → KLT

→ DFT, DCT, DWT, KLT

→ Why transforms? • computational of FFT

• Non filtering

• compression

• sparsity

1D → 2D → 3D  
(video) (video) → data input compression

Basic image depend on the ip

1. To build a test this data dependent & input transform & ST KLT is optimal

2. To use Kronecker product as a vehicle to systematically go from low dimension and high dimension. 1D → 2D → 3D DFT

3. To show study the diagonalization capacity of unitary transform

2.1.2

1D Unitary Transform

unitary  $A^H A = A A^H = I = A^H A^H A$

$$u = \begin{bmatrix} u(0) \\ u(1) \\ \vdots \\ u(N-1) \end{bmatrix}_{N \times 1} \quad y = A u \quad y = \begin{bmatrix} y(0) \\ y(1) \\ \vdots \\ y(N-1) \end{bmatrix}_{N \times 1}$$

Transform unitary

orthonormal basis

$$\begin{bmatrix} y(0) \\ y(1) \\ \vdots \\ y(N-1) \end{bmatrix} = \begin{bmatrix} a(0,0) & a(0,1) & \dots & a(0,N-1) \\ a(1,0) & a(1,1) & \dots & a(1,N-1) \\ \vdots & \vdots & \ddots & \vdots \\ a(N-1,0) & a(N-1,1) & \dots & a(N-1,N-1) \end{bmatrix} \begin{bmatrix} u(0) \\ u(1) \\ \vdots \\ u(N-1) \end{bmatrix}$$

$$\langle f, g \rangle = \sum_{i=0}^{N-1} f(i) g^*(i)$$

$$\|y\|_2^2 = y^H y = u^H A^H A u = \|u\|_2^2 \rightarrow \text{Norm preserving}$$

$$u = A^H y = A^H A u = \sum_{i=0}^{N-1} \langle u, e_i \rangle e_i^H \rightarrow \text{Parseval's theorem}$$

$$\langle e_k^H, e_l^H \rangle = \begin{cases} 1 & k=l \\ 0 & \text{otherwise} \end{cases}$$

$$\|y\|_2^2 = y^H y = \sum_{n=0}^{N-1} u(n) a^*(k,n) = \sum_{n=0}^{N-1} u(n) (a^*(k,n))^* = \langle u, A e_k^H \rangle$$

$$A^H = \begin{bmatrix} a^*(0,0) & a^*(0,1) & \dots & a^*(0,N-1) \\ a^*(1,0) & a^*(1,1) & \dots & a^*(1,N-1) \\ \vdots & \vdots & \ddots & \vdots \\ a^*(N-1,0) & a^*(N-1,1) & \dots & a^*(N-1,N-1) \end{bmatrix}$$

$\uparrow \quad \quad \quad \uparrow \quad \quad \quad \uparrow$   
 $e_k^H \quad \quad \quad e_l^H \quad \quad \quad e_m^H$

Moving from 1D to 2D

→ Frobenius norm of  $A$  matrix  $\|A\|_F = \sqrt{\sum_{i,j} |A_{i,j}|^2}$

→ Series product of 2 matrices  $\langle A, B \rangle = \sum_{i,j} A_{i,j} B_{i,j}^*$   
 $A \cdot B = \langle A, B \rangle = \|A\|_F^2$

→ Construct matrix  $A_{k,m}^* = \sum_{n=0}^{N-1} e_k^H e_m^H$

$$A = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} \quad A^H = \begin{bmatrix} a_{00}^* & a_{10}^* \\ a_{01}^* & a_{11}^* \end{bmatrix} \quad A_{k,m}^* = \begin{bmatrix} a_{00}^* & a_{10}^* \\ a_{01}^* & a_{11}^* \end{bmatrix} \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix}$$

$$A_{k,m}^* = A_{k,m}^H A_{k,m} = \begin{bmatrix} a_{00}^* & a_{10}^* \\ a_{01}^* & a_{11}^* \end{bmatrix} \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix}$$

$\langle A_{k,m}^H, A_{l,n}^H \rangle = \begin{cases} 1 & k=m, l=n \\ 0 & \text{otherwise} \end{cases}$   
2D basis ( $N^2$  matrices)  
→ orthonormal in terms of inner product

$$\langle A_{k,m}^H, A_{l,n}^H \rangle = (a_{k,m}^H)^* a_{l,n}^H = (a_{m,k}^H)^* a_{n,l}^H = (a_{m,k}^H)^* a_{n,l}^H = \delta_{k,l} \delta_{m,n}$$

$$\langle A_{k,m}^H, A_{k,m}^H \rangle = (a_{k,m}^H)^* a_{k,m}^H = (a_{m,k}^H)^* a_{m,k}^H = \delta_{m,k} \delta_{k,m} = \delta_{m,m} = 1$$

(Kronecker product) → 1D to 2D of matrices

→ 2D matrix

$$U = \sum_k \sum_l v(k, l) A_{k, l}^{\dagger} A_{k, l}$$

$$v(k,0) = \langle v, A_{k,0}^* \rangle \Rightarrow v(k,0) = \sum_m \sum_n a(k,m) u(m,n) a(l,n) \quad 0 \leq k, L \leq N-1$$

$$u(m, n) = \sum_k \sum_l A^T(k, m) v(k, l) A^T(l, n) \quad 0 \leq m, n \leq N-1$$

$$U = \begin{bmatrix} \text{अभिहित} & \text{अभिहित} \\ \text{अभिहित} & \text{अभिहित} \end{bmatrix}$$

→  $V = AUA^T \Rightarrow U = A^+VA^+ \rightarrow$  can't see from Archy (like in 1D case)

Q3. Let a 2D UT be given by  $A = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$  and the image is  $U = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ . Find transformed image & the base image.

$$A^* = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad A^* = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad A^* = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$A_{00} = \alpha_0 \alpha_0^T = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$A_{01} = \alpha_0 \alpha_1^T = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 2 & 0 \\ 2 & 0 \end{bmatrix}$$

$$A_{02} = \alpha_0 \alpha_2^T = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 2 & 2 \\ 0 & 2 \end{bmatrix}$$

$$A_{03} = \alpha_0 \alpha_3^T = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 0 & -2 \\ 0 & -2 \end{bmatrix}$$

$$V(0,0) = \langle \psi, A_{00} \rangle = \left\langle \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \right\rangle$$

$$= \frac{1}{2} \cdot 2 \cdot (1+2) = 3$$

$$V(0,1) = \langle \psi, A_{01} \rangle = \left\langle \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \frac{1}{2} \begin{bmatrix} 2 & 0 \\ 2 & 0 \end{bmatrix} \right\rangle$$

$$= \frac{1}{2} \cdot 2 \cdot (2+0) = 2$$

$$V(0,2) = \langle \psi, A_{02} \rangle = \left\langle \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \frac{1}{2} \begin{bmatrix} 2 & 2 \\ 0 & 2 \end{bmatrix} \right\rangle$$

$$= \frac{1}{2} \cdot 2 \cdot (2+4) = 6$$

$$V(0,3) = \langle \psi, A_{03} \rangle = \left\langle \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \frac{1}{2} \begin{bmatrix} 0 & -2 \\ 0 & -2 \end{bmatrix} \right\rangle$$

$$= \frac{1}{2} \cdot 2 \cdot (-2-4) = -6$$

$$V = A V A^T = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$U = A^{-1}VA^T$$

$$= \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 5 & -7 \\ -2 & 6 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$= \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 4 & 6 \\ -4 & -2 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 4 & 0 \\ 0 & -2 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & -1 \end{bmatrix}$$

### Kronecker Product of Two matrices:

$$\text{Angen} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \quad D_{\text{Angen}} = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix}$$

→ Kronecker product:  $A \otimes B = \begin{bmatrix} a_{11}B & a_{12}B \\ a_{21}B & a_{22}B \end{bmatrix}_{m \times n}$   $\neq B \otimes A$  (in general)

Page 1 of 2

$$1) (A+B) \otimes C = A \otimes C + B \otimes C$$

9)  $(A \otimes B)^T = A^T \otimes B^T$

$$2) (A \otimes B) \cdot (C \otimes D) = (AC) \otimes (BD)$$

6)  $(A \otimes B)^{-1} = A^{-1} \otimes B^{-1} \rightarrow A, B$  invertierbar

3)  $A \otimes B \otimes C = A \otimes (B \otimes C)$

$$7) \text{rank}(A \otimes B) = \text{rank}(A) \cdot \text{rank}(B)$$

$$4) \alpha(A \otimes B) = \alpha A \otimes B = A \otimes \alpha B$$

\* Examine  $A \cup B^T = V$

$A, U, G$  are arbitrary matrices

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} \quad U = \begin{bmatrix} u_{11} & u_{12} \\ u_{21} & u_{22} \end{bmatrix} \quad V = \begin{bmatrix} v_{11} & v_{12} \\ v_{21} & v_{22} \end{bmatrix}$$

Let  $v_k$  &  $v_m$  be the  $k^{\text{th}}$  &  $m^{\text{th}}$  rows of  $V \in \mathbb{R}^{n \times d}$ .

$$V^T \cdot S U^T A^T$$

$$\sqrt{\gamma} \begin{bmatrix} v_{xx} & v_{xy} \\ v_{xy} & v_{yy} \end{bmatrix} = \frac{1}{2} \begin{bmatrix} \frac{\partial u}{\partial x} & u_{xx} \\ \frac{\partial u}{\partial y} & u_{xy} \end{bmatrix} \begin{bmatrix} \frac{\partial u}{\partial x} & \frac{\partial u}{\partial y} \\ \frac{\partial u}{\partial x} & \frac{\partial u}{\partial y} \end{bmatrix} = \frac{1}{2} \begin{bmatrix} \frac{\partial u}{\partial x} \frac{\partial u}{\partial x} + \frac{\partial u}{\partial y} \frac{\partial u}{\partial y} & \frac{\partial u}{\partial x} \frac{\partial u}{\partial y} + \frac{\partial u}{\partial y} \frac{\partial u}{\partial x} \\ \frac{\partial u}{\partial x} \frac{\partial u}{\partial x} + \frac{\partial u}{\partial y} \frac{\partial u}{\partial y} & \frac{\partial u}{\partial x} \frac{\partial u}{\partial y} + \frac{\partial u}{\partial y} \frac{\partial u}{\partial x} \end{bmatrix}$$



$$\Rightarrow \underline{y}^T = B \sum_{m=0}^{N-1} \sin \omega_m T = \sum_{m=0}^{N-1} \sin \omega_m T \underline{b}^T$$

$$\text{III}^4 \quad \underline{y}_1^T = \sum_{m=0}^{N-1} \sin \omega_m T \underline{u}_m^T$$

$$\left. \begin{aligned} &\Rightarrow \left[ \begin{matrix} \underline{y}_1^T \\ \underline{y}_2^T \end{matrix} \right] = \left[ \begin{matrix} \sin \omega_0 T & \sin \omega_1 T \\ \sin \omega_2 T & \sin \omega_3 T \end{matrix} \right] \left[ \begin{matrix} \underline{b}^T \\ \underline{u}^T \end{matrix} \right] \\ &\underline{y} = (A \otimes B) \underline{u} \end{aligned} \right\} \Rightarrow \left[ \begin{matrix} \underline{y}_1^T \\ \underline{y}_2^T \end{matrix} \right] = \left[ \begin{matrix} \sin \omega_0 T & \sin \omega_1 T \\ \sin \omega_2 T & \sin \omega_3 T \end{matrix} \right] \left[ \begin{matrix} \underline{b}^T \\ \underline{u}^T \end{matrix} \right]$$

$$\underline{y} = (A \otimes B) \underline{u}$$

$$\rightarrow V = AUA^T$$

$$\Rightarrow \underline{y} = (A \otimes A) \underline{u} \rightarrow (N^2 \times 2D \text{ u.s. } N^2)$$

throughout the same decoupling change.

$$\underline{A} = A \otimes A \rightarrow 2D \text{ basis comes from Kronecker product}$$

$$\rightarrow \underline{A}^H = (A \otimes A)^H = A^H \otimes A^H$$

$$(\underline{A}^H)^T = (A^H \otimes A^H)^T = (A^H)^T \otimes (A^H)^T = A \otimes A = \underline{A}$$

$$\rightarrow \underline{y} \cdot \underline{A} \underline{u} \Rightarrow \underline{u} + \underline{A}^H \underline{y}$$

2D basis: cols of  $\underline{A}^H$

$$\underline{u} = \underline{A}^H \underline{y}$$

Ex 1

$$\rightarrow \underline{A} = A \otimes A = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \otimes \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$\frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

$$\underline{A}^H = \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

$$\underline{y} = \underline{A}^H \underline{u}$$

$$\Rightarrow \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix} \begin{bmatrix} \underline{u} \\ \underline{v} \end{bmatrix} = \begin{bmatrix} 10 \\ -2 \\ -4 \\ 8 \end{bmatrix} = \begin{bmatrix} 5 \\ -1 \\ -2 \\ 4 \end{bmatrix}$$

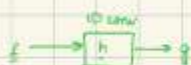
$$\underline{u} = \begin{bmatrix} 5 \\ -1 \\ -2 \\ 4 \end{bmatrix}$$

1D DFT:

$$\left. \begin{aligned} \underline{f}(n), \quad 0 \leq n \leq N-1 \\ \underline{F}(k), \quad 0 \leq k \leq N-1 \\ \underline{f}(n) = \frac{1}{N} \sum_{k=0}^{N-1} \underline{F}(k) e^{j \frac{2\pi}{N} kn} \\ \underline{F}(k) = \sum_{n=0}^{N-1} \underline{f}(n) e^{-j \frac{2\pi}{N} kn} \end{aligned} \right\} \quad 0 \leq k, n \leq N-1$$

$$\underline{F} = \underline{\Phi} \underline{f} \quad \underline{\Phi}_{N \times N} \quad \underline{\Phi}_{mn} = e^{-j \frac{2\pi}{N} mn}$$

$$\underline{f} = \underline{\Phi}^H \underline{F} \quad \underline{\Phi} = \frac{1}{\sqrt{N}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad \underline{\Phi}^H \underline{\Phi} = \underline{I} \quad \underline{\Phi} \underline{\Phi}^H = \underline{I}$$



$$\underline{g} = \underline{f} * \underline{h}$$

$$\underline{g} = \underline{H} \underline{f} \quad \text{Discrete DFT}$$

$$\underline{\Phi} \underline{g} = \underline{\Phi} \underline{H} \underline{f}$$

$$= \underline{\Phi} \underline{H} \underline{\Phi}^H \underline{\Phi} \underline{f}$$

$$\underline{G} = \underline{\Phi} \underline{H} \underline{\Phi}^H \underline{F}$$

diagonalization

$$\underline{H} \rightarrow \text{DFT of } \underline{h}$$

RD-DFT:  $\rightarrow$  Separate transform by construction

$$\underline{F}(k, \ell) = \frac{1}{N} \sum_{m=0}^{N-1} \sum_{n=0}^{N-1} \underline{f}(m, n) e^{-j \frac{2\pi}{N} (k m + \ell n)} \quad 0 \leq k, \ell \leq N-1$$

$$e^{-j \frac{2\pi}{N} k m} e^{-j \frac{2\pi}{N} \ell n}$$

$$A(m, k, N, L)$$

$$\downarrow$$

$$C(m, k) B(n, \ell)$$

$$\underline{\Phi}^{-1} = \underline{\Phi}^H \otimes \underline{\Phi}^H$$

$$= \underline{\Phi}^H \otimes \underline{\Phi}^H$$

$$= (\underline{\Phi} \otimes \underline{\Phi})^H$$

$$\underline{\Phi}^{-1} = \underline{\Phi}^H$$

$$\rightarrow \underline{\Phi} = \underline{\Phi} \otimes \underline{\Phi} \Rightarrow \underline{y} = \underline{\Phi} \underline{u}$$

$$\underline{u} = \underline{\Phi}^H \underline{y}$$

$\rightarrow$  Instead of  $\underline{H}$ , consider covariance matrix

'R' which is circulant

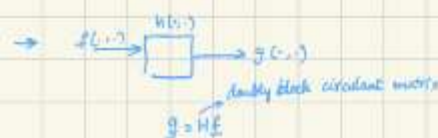
$$\underline{\Phi} \underline{R} \underline{\Phi}^H$$

(decoupling)

$\rightarrow$  DFT & circulant matrix have special affinity. [ $\underline{\Phi} \underline{H} \underline{\Phi}^H \rightarrow$  diagonalization]

$\rightarrow$  Every normal matrix is unitarily diagonalizable.

→ DCT can almost diagonalize Markov (Luto)



→ 2D DFT diagonalizes DBC matrix.

$$\begin{aligned} \Phi G &= \Phi H F \\ \underline{G} &= \underbrace{\Phi H \Phi^*}_\text{diagonalization} F \end{aligned}$$

$$\underline{G} = \Delta_H F$$

→ 2D DFT



$$\underline{G} = (\Phi_{\text{max}} \otimes \Phi_{\text{max}} \otimes \Phi_{\text{col}}) \frac{1}{N} \downarrow$$

Diagram showing a 2D grid with axes  $k$  and  $n$ , representing the 2D DFT.



→ 1D DFT shows how  $f_n$  varies across  $x$

2D DFT shows how intensity varies across  $x$  &  $y$

→ For  $k \in \frac{N}{2}$   $f(n) = e^{j\frac{2\pi}{N}kn}$  the sign wiggles fastly (value)

Mag. phase dominance

→ Phase component change is dominant in transform.

→ The phase of image spectrum has a greater effect in the image reconstruction process than its magnitude.

2D DFT Computation:

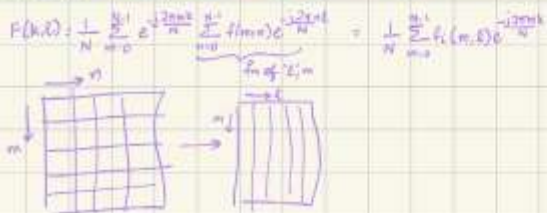
$$F(k, l) = \frac{1}{N} \sum_{m=0}^{N-1} \sum_{n=0}^{N-1} f(m, n) e^{-j\frac{2\pi}{N}(km + ln)}$$

Diagram showing a 2D grid with axes  $k$  and  $l$ , representing the 2D DFT.

1D DFT  $\rightarrow O(N^2)$   
FFT  $\rightarrow O(N \log N)$

→ Row: col-by-col decomposition  $\rightarrow O(N(N \log N))$

$$\begin{aligned} \rightarrow \text{Same for } V &= A U^T \\ &= A (U^T)^T \end{aligned}$$



\* What is the order if img is  $M \times N$ ?

Rotation property:

→  $f$  is rotated by  $\theta$ . →  $|f|$  also gets rotated by  $\theta$ .

1D DCT (Discrete cosine transform) → used more compared to DFT & FFT

→ Used in jpeg, mp3, H.263

- DCT concentrates energy in the first few coefficients (energy packing). (Deterministic)
- <sup>gradient</sup> Better decorrelation for strongly correlated for Markov process. (Statistical)

→ DCT avoids spectral leakage

1D DCT-2

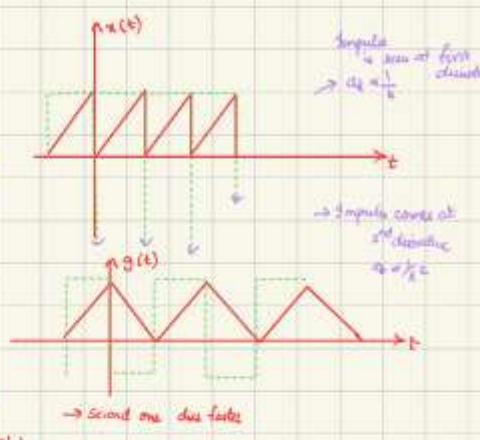
$$v(k) = \alpha(k) \sum_{n=0}^{N-1} u(n) \cos\left(\frac{\pi(2n+1)k}{2N}\right) \quad 0 \leq k \leq N-1$$

$$u(n) = \sum_{k=0}^{N-1} \alpha(k) v(k) \cos\left[\frac{\pi(2n+1)k}{2N}\right] \quad 0 \leq n \leq N-1$$

$$V = CU$$

$$C(k,n) = \alpha(k) \cos\left[\frac{\pi(2n+1)k}{2N}\right] \quad 0 \leq k, n \leq N-1$$

$$C(k,n) = \begin{cases} \frac{1}{\sqrt{N}} & k=0 \\ \sqrt{\frac{2}{N}} \cos\left(\frac{\pi(2n+1)k}{2N}\right) & 1 \leq k \leq N-1 \end{cases}$$



$$\alpha(k) = \begin{cases} \frac{1}{\sqrt{N}} & k=0 \\ \sqrt{\frac{2}{N}} & 1 \leq k \leq N-1 \end{cases}$$

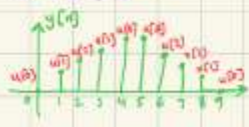
$N=5$

$$C = \begin{bmatrix} \frac{1}{\sqrt{5}} & \frac{1}{\sqrt{5}} & \frac{1}{\sqrt{5}} & \frac{1}{\sqrt{5}} & \frac{1}{\sqrt{5}} \\ \sqrt{\frac{2}{5}} \cos\frac{\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{2\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{3\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{4\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{5\pi}{5} \\ \sqrt{\frac{2}{5}} \cos\frac{2\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{4\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{6\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{8\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{10\pi}{5} \\ \sqrt{\frac{2}{5}} \cos\frac{3\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{6\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{9\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{12\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{15\pi}{5} \\ \sqrt{\frac{2}{5}} \cos\frac{4\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{8\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{12\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{16\pi}{5} & \sqrt{\frac{2}{5}} \cos\frac{20\pi}{5} \end{bmatrix}$$

$C^T = C^{-1}$   
 $C^H C = I$   
 $C^H = C^T$

Rel to DFT:

$$u[n] \quad 0 \leq n \leq N-1$$



Let us construct seq.

$$y[n] = \begin{cases} u[n] & 0 \leq n \leq N-1 \\ u[N-n] & N \leq n \leq 2N-1 \end{cases}$$

Take 2N pt DFT of  $y[n]$

$$Y[k] = \frac{1}{\sqrt{2N}} \sum_{n=0}^{2N-1} y[n] e^{-j\frac{2\pi}{2N}kn}$$

$$= \frac{1}{\sqrt{2N}} \sum_{n=0}^{N-1} u[n] e^{-j\frac{2\pi}{2N}kn} + \frac{1}{\sqrt{2N}} \sum_{n=N}^{2N-1} u[2N-n] e^{-j\frac{2\pi}{2N}kn}$$

$$= \frac{1}{\sqrt{2N}} \sum_{n=0}^{N-1} u[n] e^{-j\frac{2\pi}{2N}kn} + \frac{1}{\sqrt{2N}} \sum_{n=0}^{N-1} u[n] e^{-j\frac{2\pi}{2N}k(2N-n)}$$

$$= \frac{1}{\sqrt{2N}} \sum_{n=0}^{N-1} u[n] e^{-j\frac{2\pi}{2N}kn} + \frac{1}{\sqrt{2N}} \sum_{n=0}^{N-1} u[n] e^{-j\frac{2\pi}{2N}k(2N-n)}$$

$$= \frac{1}{\sqrt{2N}} \sum_{n=0}^{N-1} u[n] e^{-j\frac{2\pi}{2N}kn} + \frac{1}{\sqrt{2N}} \sum_{n=0}^{N-1} u[n] e^{-j\frac{2\pi}{2N}k(2N-n)}$$

$$= \frac{1}{\sqrt{2N}} \sum_{n=0}^{N-1} u[n] e^{-j\frac{2\pi}{2N}kn} + \frac{1}{\sqrt{2N}} \sum_{n=0}^{N-1} u[n] e^{-j\frac{2\pi}{2N}k(2N-n)}$$



$$Y(k) = \frac{1}{\sqrt{2N}} e^{j\frac{\pi k}{2N}} \sum_{n=0}^{N-1} u[n] e^{-j\frac{\pi kn}{2N}} e^{j\frac{\pi kn}{2N}} = u[n] e^{j\frac{\pi kn}{2N}} e^{j\frac{\pi kn}{2N}}$$

$$= \frac{1}{\sqrt{2N}} e^{j\frac{\pi kn}{2N}} \sum_{n=0}^{N-1} u[n] \left( e^{-j\frac{\pi kn}{2N}} + e^{j\frac{\pi kn}{2N}} \right)$$

$$= \frac{1}{\sqrt{N}} e^{j\frac{\pi kn}{2N}} \sum_{n=0}^{N-1} u[n] \cos\left(\frac{\pi kn}{2N}\right)$$

→ Instead of (2N-pt) DFT to get DCT, we can do a (N-pt) DFT.

$$Y(k) = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} y(n) e^{-j\frac{\pi kn}{N}}$$

$$= \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} \underbrace{y(2n)}_{y_e[n]} e^{-j\frac{\pi k(2n)}{N}} + \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} \underbrace{y(2n+1)}_{y_o[n]} e^{-j\frac{\pi k(2n+1)}{N}}$$

Diagonalization: (Statistical sense)

→ DFT can diagonalize LTI system which is circulant & decoupled a correlation matrix which is circulant.

→  $\text{DFT}$   
 $\text{CRCT} \neq \Lambda_R$   
 circulant

→ For what  $R$ ,  $\text{CRCT} = \Lambda_R$

$R \rightarrow$  Markov-1 process (i.e.  $P(x_n | x_{n-1}, x_{n-2}, \dots, x_1) = P(x_n | x_{n-1})$ )

$0 < \rho < 1$

→ Toeplitz

$$R = \begin{bmatrix} 1 & \rho & \rho^2 & \rho^3 \\ \rho & 1 & \rho & \rho^2 \\ \rho^2 & \rho & 1 & \rho \\ \rho^3 & \rho^2 & \rho & 1 \end{bmatrix} \rightarrow \text{Almost diagonalizable for } \rho \sim 1$$

$\beta^2 R^{-1} = \begin{bmatrix} 1-\rho & -\rho & 0 & 0 \\ -\rho & 1 & -\rho & 0 \\ 0 & -\rho & 1 & -\rho \\ 0 & 0 & -\rho & 1-\rho \end{bmatrix}$  where  $\beta^2 = \frac{1-\rho^2}{1+\rho^2}$

$\alpha = \frac{\rho}{1+\rho^2}$

If  $B = \begin{bmatrix} 1-\alpha & -\alpha & 0 & 0 \\ -\alpha & 1 & -\alpha & 0 \\ 0 & -\alpha & 1 & -\alpha \\ 0 & 0 & -\alpha & 1-\alpha \end{bmatrix}$  not same as  $\beta^2 R^{-1}$  in  $\mathbb{R}^4$

$\text{CBCT} = \Lambda_B$

$B = \begin{bmatrix} 1-\alpha & -\alpha & 0 & 0 \\ -\alpha & 1 & -\alpha & 0 \\ 0 & -\alpha & 1 & -\alpha \\ 0 & 0 & -\alpha & 1-\alpha \end{bmatrix} = \begin{bmatrix} \alpha & 0 & 0 & 0 \\ 0 & \alpha & 0 & 0 \\ 0 & 0 & \alpha & 0 \\ 0 & 0 & 0 & \alpha \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Toeplitz Eigen Pair

→ If Markov-1 process has higher  $\rho$  (say 0.95, 0.9)

$R^{-1}$  looks like  $B$

⇒ DCT almost diagonalize

highly correlated Markov process

Walsh-Hadamard Transform (WHT):

$\underline{y} = H \underline{x}$   
 $\underline{x} = H^{-1} \underline{y}$

$H^{-1} = H$

The Hadamard matrices  $H_N$  are  $N \times N$  matrices where  $N = 2^n$ ,  $n = 1, 2, 3, \dots$

$H_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$

$H_2 = \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$

$H_n = H_{n-1} \otimes H_1$

$\downarrow$   
 $2^n \times 2^n$

Data dependent transform:



$V = AU$

'A' depends on 'u'

KLT - Karhunen-Loève Transform  $\rightarrow$  Principal components analysis (PCA)

Spectral theorem: Any normal matrix is unitarily diagonalizable.

$R \gg N$

$X = \begin{bmatrix} | & | & | \\ \vdots & \vdots & \vdots \\ | & | & | \end{bmatrix}_{N \times P}$

$\hat{u} = \frac{1}{P} \sum_{p=1}^P u_p$   
covariance  $\hat{R} = \frac{1}{P} \sum_{p=1}^P (u_p - \hat{u})(u_p - \hat{u})^T$   
 $R = (X - U)(X - U)^T$

$\rightarrow \Psi^H R \Psi = \Lambda$

Need to find  $\Psi$  which diagonalize  $R$ .

$\Psi$  exists (Spectral theorem)

$\Psi^H R \Psi = \Lambda \Rightarrow R = \Psi \Lambda \Psi^H$

$R \Psi_i = \Psi_i \Lambda_{ii}$

$= \Psi_i \begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_N \end{bmatrix} \begin{bmatrix} \psi_{i1} \\ \vdots \\ \psi_{iN} \end{bmatrix}$

$= \Psi_i \begin{bmatrix} \lambda_i \\ 0 \\ \vdots \end{bmatrix}$

$R \Psi_i = \lambda_i \Psi_i$

25/10

$\rightarrow R^T = U R U^H \Rightarrow \|R\|_F^2 = \text{trace}(R^T R)$   
 $= \text{trace}(U R U^H U R U^H)$   
 $= \text{trace}(R^T R) = \text{trace}(\Lambda^T \Lambda)$

$\rightarrow$  Energy packing efficiency

$$\text{EPF}(R) = \frac{\sum_{i=1}^N \lambda_i^2(1)}{\sum_{i=1}^N \lambda_i^2(i)}$$

$\rightarrow$  we want this to be high

$\rightarrow$  Decoulation efficiency  $\eta = 1 - \frac{\alpha}{\beta}$

$\alpha = \sum_{\substack{i,j=1 \\ i \neq j}}^N |\lambda_i \lambda_j|$

$\beta = \sum_{i,j=1}^N |\lambda_i \lambda_j|$

After transformation, if it is close to diagonalizable  
off diagonal elements close to zero.  
Perfectly diagonalizable  $\rightarrow \alpha=0$ .

If we restrict to  $P$  out of the eigen basis

$\hat{y} = \Psi(\hat{u} - \hat{u})$

$\hat{u} = \Psi^H \hat{y} + \hat{u}$

$\hat{m} = \frac{1}{P} \sum_{p=1}^P u_p$

$\Psi = \begin{bmatrix} | & | & | \\ \vdots & \vdots & \vdots \\ | & | & | \end{bmatrix}_{N \times P}$

Ex. 2

$E[\hat{y} \hat{y}^H] \in E[\Psi(\hat{u} - \hat{u})(\hat{u} - \hat{u})^H \Psi^H]$   
 $= \Psi R \Psi^H = \Lambda_R$

$\rightarrow$  So instead of  $u$ , we will have  $\hat{u} = \Psi_{N \times P}^H \frac{1}{P} \sum_{p=1}^P u_p + \hat{m}$

$\rightarrow$  Error  $\|\hat{u} - \hat{u}\|_F^2 = \sum_{i=P+1}^N \lambda_i^2$   
(on avg)  $\rightarrow$  var & non-var  
Statistics (No eigen value of 'R' - covariance matrix)

for sending info  
 $\rightarrow$  KLT is not used much because basis keeps changing as it is data dependent.

- $\rightarrow$  It sets a benchmark "if it comes close to data in dept."
- $\rightarrow$  Used for visualization
- $\rightarrow$  Eigen recognition
- $\rightarrow$  Eigen filtering.
- $\rightarrow$  Dimensionality reduction

## Singular Value Decomposition (SVD)

1. Generalization of the notion of diagonalization of the rectangular matrix
2. Interpret as a data-split unitary transform (non-statistical sense)
3. Best way to:

$$S_{m \times n} = A \Sigma B^H \quad (\text{General notation})$$

$\rightarrow$  singular values of  $S$

$$\rightarrow S_{3 \times 2} = A_{3 \times 3} \Sigma_{3 \times 2} B_{2 \times 2}^H$$

$$A \text{ \& } B \text{ are unitary matrices i.e. } AA^H = A^H A = I_{3 \times 3}, BB^H = B^H B = I_{2 \times 2}$$

$$\rightarrow \Sigma_{3 \times 2} = \begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \end{bmatrix}$$

$\rightarrow$  Not diagonal matrix (but off diagonal = 0)

$\sigma_1, \sigma_2$  - singular values

$$\rightarrow S_{m \times n} \rightarrow \text{rank} - P \quad (P \leq \min(m, n))$$

$$S_{m \times n} = A \Sigma B^H$$

$\rightarrow \sum_{i=1}^P \sigma_i (u_i v_i^H)$   $\rightarrow$  Basis image

$$\rightarrow E = S \cdot \hat{S} = \sum_{i=1}^P \sigma_i (a_i b_i^H)$$

$$E_{mm} = \sum_{i=1}^P \sigma_i (a_i m^H b_i)$$

$$E_{nn}^* = \sum_{i=1}^P \sigma_i (a_i m^H b_i)$$

$$\begin{aligned} (E_{nn})^2 &= E_{nn}^* E_{nn} \\ &= \sum_{i=1}^P \sum_{j=1}^P \sigma_i \sigma_j a_i m^H b_i a_j m^H b_j \\ &= \sum_{i=1}^P \sigma_i^2 a_i m^H a_i m^H b_i b_i^H \\ &\quad + \sum_{i \neq j} \sigma_i \sigma_j a_i m^H a_j m^H b_i b_j^H \end{aligned}$$

$$\begin{aligned} \sum_{m=1}^n |E_{nn}|^2 &= \sum_{m=1}^n \sum_{i=1}^P \left( \sigma_i^2 \sum_{j=1}^P |a_i m^H a_j m^H b_i b_j^H| \right) \\ &= \sum_{i=1}^P \sigma_i^4 \sum_{m=1}^n |a_i m^H a_i m^H b_i b_i^H| \\ &\quad + \sum_{i \neq j} \sigma_i^2 \sigma_j^2 \sum_{m=1}^n |a_i m^H a_j m^H b_i b_j^H| \end{aligned}$$

$$\begin{bmatrix} A^H A & A^H B \\ B^H A & B^H B \end{bmatrix}$$

$\|S - \hat{S}\|_F^2 \rightarrow$  Frobenius norm of  $S$

$$\Rightarrow \sum_{m=1}^n \sum_{i=1}^P (E_{nn})^2 = \sum_{i=1}^P \sigma_i^4$$

$\rightarrow$  SVD gives the best low-rank approximation matrix

$\rightarrow$  SVD is closely related to PCA

$$\underline{X} = \begin{bmatrix} | & | & \dots & | \end{bmatrix}_{N \times P}$$

$P \gg N$

$$\bar{X}_{\text{avg}} = X - \underline{1}$$

$\rightarrow$  avg of all the cols

$$\bar{X} \bar{X}^T = A \Sigma B^H B \Sigma A^H$$

$$\bar{X} \bar{X}^T B^H = A \Sigma \Sigma^H A^H$$

$\rightarrow$   $A$  needed for PCA is the same  $A$  which can be obtained from SVD of  $\bar{X}$

## Pseudo inverse (Weak inverse)

$$S_{\text{min}} = A \Sigma B^H$$

$$\rightarrow S S^H = S$$

$$S_{\text{min}}^+ = B \Sigma^+ A^H$$

$$S S^+ = S^+$$

$$\Sigma^+ = \begin{bmatrix} \frac{1}{\sigma_1} & 0 & 0 \\ 0 & \frac{1}{\sigma_2} & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$$\Sigma \Sigma^+ = I$$

but  $S S^+ \neq I$   
that's why it is called pseudo inverse



# 11.3

not any remain before

## IMAGE ENHANCEMENT (IE): (Subject)

→ Improving the visual appeal of an image

Eg. contrast, illumination, segmentation, edges (only edge detect)  
 ↳ then we get edges in only pixel around a diff. pixel

→ Change detection, edge detection → requires thresholding →  $\text{diff}(i,j) = |I_1(i,j) - I_2(i,j)|$   
 If  $\text{diff}(i,j) > T$ ,  $\text{out}(i,j) = 1$   
 else  $\text{out}(i,j) = 0$

→ Histogram

Class thresholding:

→ Mean & var of each class

$$u_1(t) = \frac{\sum_{i=0}^L i \cdot F(i)}{N_1} \rightarrow \text{pixels in class 1}$$

$$r_1^2(t) = \frac{\sum_{i=0}^L (i - u_1(t))^2 \cdot F(i)}{N_1}$$

$$u_2(t) = \frac{\sum_{i=L+1}^L i \cdot F(i)}{N_2}$$

$$r_2^2(t) = \frac{\sum_{i=L+1}^L (i - u_2(t))^2 \cdot F(i)}{N_2}$$



Define within-class var. to be

$$\sigma_w^2(t) = \frac{\sum_{j=1}^2 \sigma_j^2(t)}{N} \rightarrow N = N_1 + N_2$$

Between class var

$$\sigma_b^2(t) = \frac{\sum_{j=1}^2 (u_j(t) - u_T)^2 \cdot N_j}{N}$$

Total mean / var

$$u_T = \frac{\sum_{i=0}^L i \cdot F(i)}{N}$$

$$\sigma_T^2 = \frac{\sum_{i=0}^L (i - u_T)^2 \cdot F(i)}{N}$$

↓  
 $\sigma_T^2 \propto (u_1(t) - u_2(t))^2$   
 It can be shown that  
 $\sigma_T^2 = \sigma_w^2(t) + \sigma_b^2(t)$   
 (variance of sum)

$$\begin{aligned} \sigma_T^2 &= \frac{\sum_{i=0}^L (i - u_T)^2 \cdot F(i)}{N} = \frac{\sum_{i=0}^L (i - u_T)^2 \cdot F(i)}{N} + \frac{\sum_{i=L+1}^L (i - u_T)^2 \cdot F(i)}{N} \\ &= \frac{\sum_{i=0}^L (i - u_1 + u_1 - u_T)^2 \cdot F(i)}{N} \\ &= \frac{\sum_{i=0}^L [(i - u_1)^2 + (u_1 - u_T)^2 + 2(i - u_1)(u_1 - u_T)] \cdot F(i)}{N} \\ &= \frac{N_1}{N} \frac{\sum_{i=0}^L (i - u_1)^2 \cdot F(i)}{N_1} + \frac{N_1}{N} (u_1 - u_T)^2 \frac{\sum_{i=0}^L F(i)}{N_1} + 2(u_1 - u_T) \frac{\sum_{i=0}^L (i - u_1) \cdot F(i)}{N} \\ &= \frac{N_1}{N} \sigma_1^2(t) + \frac{N_1}{N} (u_1(t) - u_T)^2 + \frac{N_2}{N} \sigma_2^2(t) + \frac{N_2}{N} (u_2(t) - u_T)^2 \\ &= \sigma_w^2(t) + \sigma_b^2(t) \end{aligned}$$

(11)

Multi-level thresholding: Otsu

Optimal threshold:  $t^*$ , argmax  $\sigma_b^2(t)$  at each

→ Depending on the no. of classes we have as many no. of thresholds

$$\{t_1^*, t_2^*, t_3^*, \dots\} = \arg \max \sigma_b^2(t_1, t_2, t_3, \dots)$$

$$0 \leq t_1 \leq t_2 \leq t_3 \dots$$

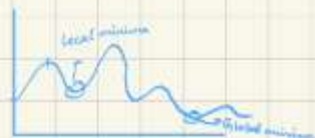
$$\text{where class } c_1 \in \{0, t_1\}, c_2 \in \{t_1, t_2\}, c_3 \in \{t_2, t_3\}$$

→ When we have m classes

$$\sigma_b^2(t_1, t_2, t_3, \dots) = \frac{\sum_{j=1}^m (u_j(t_1, t_2, t_3, \dots) - \mu_T)^2 \cdot N_j}{N}$$

K-means clustering: Other is 1-D analog of K-means

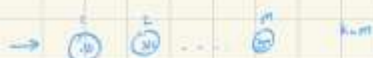
→ Greedy algorithm → Prone to local minima instead of global minima (like in EM)



→ Unsupervised

→ Guarantees local minimum

→ Grouping into clusters (K-clusters) via supervision



Similarity measure  $x_1, x_2, \dots, x_n, \mu_1, \mu_2, \dots, \mu_K$

→ Euclidean dist

$$K^* = \arg \min_{1 \leq k \leq K} \sum_{i=1}^n (x_i - \mu_k)^2$$

② Normalized Mahalanobis dist:  $x_i \in K^*$  where  $K^* = \arg \min_{1 \leq k \leq K} (x_i - \mu_k)^T R_k^{-1} (x_i - \mu_k)$

$$\text{if } (x_i - \mu_k) \text{ is scalar} \quad \frac{(x_i - \mu_k)^2}{\sigma_k^2}$$

③ EM algorithm

$$p(x) \sim N(\mu, \sigma^2) \left( \sum_{i=1}^K \left[ \frac{1}{\sigma_i^2} \exp\left(-\frac{(x - \mu_i)^2}{2\sigma_i^2}\right) \right] \right) \rightarrow \text{if } \sigma_i \rightarrow 0, \text{ it is a Dirac delta}$$

$$N(x) = \frac{1}{\sigma^2} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

Steps of K-means

$\{x_1, x_2, \dots, x_n\}$  K-means  $\{c_1, c_2, \dots, c_K\}$  → clusters

1. Obtain randomly K initial pattern centers (Pick random  $K$  from  $\{x_1, \dots, x_n\}$  & assign them as each cluster center) → next divide the dataset into  $K$  clusters by assigning each data sample to the nearest pattern center

2. Find the sum square error for all the samples

3. Recalculate the cluster centers to be the centroid of the current cluster

4. Using the current set of cluster centers, recalculate the data partition by reassigning each data sample to the nearest pattern cluster

• If the data assignment (cluster assignment) remains unchanged, then go to step 5

this go to step 5

5. Return the current set of pattern/cluster centers

Why is it greedy?

Loss fn for the update step is  $L = \sum_{i=1}^n \sum_{j=1}^K \frac{1}{n_j} \|x_i - \mu_j\|^2$

→ current assignment

$$\frac{\partial L}{\partial \mu_j} = \sum_{i \in c_j} (x_i - \mu_j) = 0$$

$$\uparrow$$

$$n_j \mu_j = \sum_{i \in c_j} x_i$$

$$\Rightarrow \mu_j = \frac{1}{n_j} \sum_{i \in c_j} x_i$$

6.4 How to fix k? Instead we can do iterative clustering

Iterative clustering (Heuristic self-organizing data clustering)

→ Also unsupervised

→ (Merge, discard, split)

Merge It merges clusters  $i$  &  $j$  if  $\|m_i - m_j\|^2 < \tau_1$   
i.e. means of 2 clusters are too close  $\Rightarrow$  can be merged.

Discard Discard a cluster if it has too few samples and assign those samples to other clusters.

Split Split a cluster if max. eigen value of the corresponding covariance is larger than  $\tau_2$ .

Trends  $\tau_1, \tau_2, \tau_3$

→ compute  $\lambda_1, \lambda_2, \dots$  (eigen value corresponding to min. eigen value)  
if  $\lambda_1 > 0$  assign  $x_i$  to one cluster  
 $\lambda_1 < 0$  assign  $x_i$  to the other cluster

→ dynamic range  $\rightarrow$  how many intensity levels does a image occupy.  
Contrast Enhancement (aka stretching)



→ when we have poor contrast, how to improve that? Histogram equalization, Histogram modification



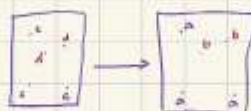
Linear contrast stretching

$$P_{out} = \frac{P_{in} - c}{(d - c)} (b - a) + a$$

$$\begin{aligned} P_{in} &= c \\ &\Rightarrow P_{out} = a \\ P_{in} &= d \\ &\Rightarrow P_{out} = b \end{aligned}$$

→  $P_{in}$  is the i/p gray level value

$a$  &  $b$  - desired min & max  
 $c$  &  $d$  - i/p



→ Better method is histogram equalization

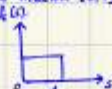
i/p histogram  $\rightarrow$  o/p dist. shd be flat

→ Consider a cont. RV  $X \geq 0$  with pdf  $f_X(x)$  [can be missing]

Has a CDF  $F_X(x) = P(X \leq x) = \int_0^x f_X(t) dt$

Goal: Transform rv  $X$  st the transformed rv  $Y$  has  $f_Y(y) = 1$  in the interval  $[0, 1]$  & '0' outside.

$$Y = T(X)$$





$$\rightarrow f_z(z) = \frac{f_y(F_z^{-1}(z))}{\left| \frac{dz}{dy} \right|_{y=F_z^{-1}(z)}}$$

$\rightarrow S: F_Y(y) \rightarrow$  Probability integral transforms (PIT)

$$\Rightarrow f_z(z) = \frac{f_y(F_z^{-1}(z))}{|f_z'(y)|_{y=F_z^{-1}(z)}} \quad \because f_z(z) > 0$$

$$= \frac{f_y(F_z^{-1}(z))}{f_z(F_z^{-1}(z))} = 1$$

histogram modification: Let  $f_Y(y)$  &  $f_Z(z)$  be i/p & target(desired) pdfs

First transform  $y$  then PIT

$$y = F_Y(y)$$

Also transform  $z$  then PIT

$$v = F_Z(z)$$

$$\Rightarrow \boxed{z = F_Z^{-1}(F_Y(y))}$$

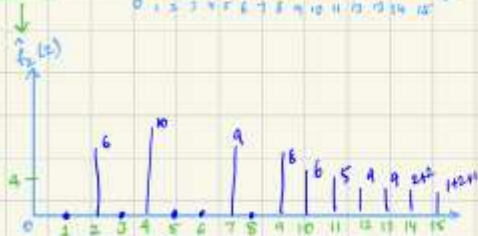
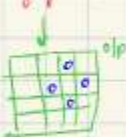
$\rightarrow$  HM = HB if  $z$  is uniform

$\rightarrow$  clearly image is discrete, so we can't guarantee if inverse cdf exists.

$\rightarrow$  So how to get cont. version of discrete histogram?

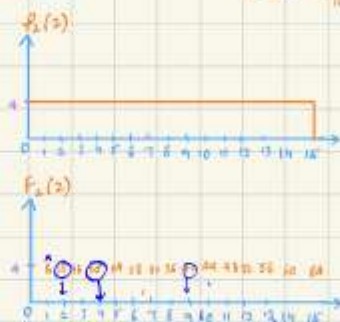
8.14

HE example:



$$z = F_Z^{-1}(F_Y(y))$$

64 pixels  
16 levels  
 $\frac{64}{16} = 4$



$\rightarrow$  Redistribution of the allocated intensity to preserve the monotonicity of CDF.

We go for next higher value in matching CDF values.

eg: 11M



## Noise fitting:

- AWGN: Signal indep (additive)
- Poisson: Signal dep
- Speckle: Multiplicative noise
- Impulse:

## AWGN filtering:

### Image sequence filtering



$$g_i(x, y) = f(x, y) + n_i(x, y) \quad \text{Determine } n_i(x, y), n_i(x, y)$$

$$\hat{f}(x, y) = \frac{1}{M} \sum_{i=1}^M g_i(x, y)$$

↓  
Ker:  $(-g \rightarrow x)$

$$E[\hat{f}(x, y)] = E[f(x, y)] = E\left[\frac{1}{M} \sum_{i=1}^M g_i(x, y)\right]$$

$$= \frac{1}{M} \sum_{i=1}^M E[g_i(x, y)]$$

$$= \frac{1}{M} \sum_{i=1}^M E[f(x, y) + n_i(x, y)]$$

$$= \frac{1}{M} [M f(x, y)] + \frac{1}{M} \sum_{i=1}^M E[n_i(x, y)]$$

$$E[\hat{f}(x, y)] = f(x, y) \quad \text{[Unbiased estimator]}$$

$$\sigma_{\hat{f}}^2(x, y) = E[(\hat{f}(x, y) - f(x, y))^2]$$

$$= E\left[\left(\frac{1}{M} \sum_{i=1}^M g_i(x, y) - f(x, y)\right)^2\right]$$

$$= E\left[\left(\frac{1}{M} \sum_{i=1}^M n_i(x, y)\right)^2\right] = \frac{1}{M^2} E\left[\sum_{i=1}^M n_i^2(x, y)\right]$$

$$= \frac{1}{M^2} \cdot M \cdot \sigma_n^2 = \frac{\sigma_n^2}{M} \Rightarrow \boxed{\sigma_{\hat{f}}^2(x, y) = \frac{\sigma_n^2}{M}} \rightarrow \text{consistent estimator}$$

As  $M \uparrow$ ,  $\sigma^2 \downarrow$

## Spatially local filters:

### Mean filter:

$$g(\underline{p}) = \frac{1}{N} \sum_{\underline{q} \in \text{neighborhood of } \underline{p}} f(\underline{q})$$

↑  
spatial filter

↓  
Normalization const. to ensure filter output sums up to 1

$$Z = \sum_{\underline{q} \in \text{neighborhood of } \underline{p}} f(\underline{q})$$

$$Bw(\underline{p}) = \begin{cases} \frac{1}{N} & \text{if } \underline{p} \in \text{neighborhood of } \underline{p} \\ 0, & \text{otherwise} \end{cases}$$

## Gaussian filter (weighted avg)

$$g(p) = \frac{1}{2} \sum_{q \in G} G_r(p, q) f(q)$$

where  $G_r(p, q) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(p-q)^2}{2\sigma^2}\right)$  is the Gaussian kernel.  $\sigma$  is the standard deviation.

10/4

## Non-Linear Means filter (NLM)

$$g(p) = \frac{1}{2} \sum_{q \in G} G_r(N_p, N_q) f(q)$$

$G \Rightarrow$  can be entire spatial domain [typical size  $= 21 \times 21$ ]

$N_p \Rightarrow$  patch centered at  $p$  (typically  $7 \times 7$ )

$N_q \Rightarrow$  patch centered at  $q$ ,  $f(q) \rightarrow$  intensity at  $q$

$\Rightarrow N_p, N_q$  compares the intensity

$\rightarrow$  When the patch reduces to a single pixel, then it is a special case of NLM filter - **Yaroslavsky filter**.

$\rightarrow$  When patch is larger, it gives better performance.

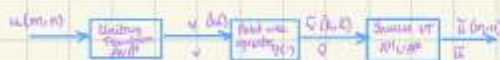
$\rightarrow$  Normalization constant  $z = \sum_{q \in G} G_r(N_p, N_q)$

$$G_r(z) = \exp\left(-\sum_{i=1}^n x_i^2 / h^2\right) \quad n \rightarrow \text{patch size}$$

$\rightarrow$  Higher 'h' will mean that even not so 'like' patches will get a reasonable weight. Reads if the patches are similar.

$\rightarrow$  Space variant filter. Edge preserving.

## Transformed Domain Filtering



$\rightarrow$  SVD can be used to denoise the image.

$$S = \sum_{i=1}^P r_i (a_i b_i^T) = A \Sigma B^T$$

$r_i$  which is large corresponding to noise

$$S = \sum_{i=1}^n r_i (a_i b_i^T)$$

## BM 3D (Block Matching 3D filter)

$\rightarrow$  Collaborative filtering (sparsity based denoising)

(i) Divide the image into patches  $T_i (8 \times 8 \times \text{max})$

( $\oplus$  is  $8 \times 8$  block we use DCT)

(ii) Compare it with other patches & find similar patches using some threshold. Group patches by comparing patch intensity.



10) Form a 3D block

11) Apply a suitable 3D unitary transform (DCT/DFT) on the 3D block

12) Shrinkage: Do hard thresholding

13) Apply inverse 3D VT on the shrunk coefficients

14) Put the patches back in the locations where they came

15) Inverse weight a patch depending upon the # of non-zero coefficients used to do weighted averaging

15A)

## Image restoration: (Objective)

→ Restoration is usually inverse problems

Undoing degradation

→ enhancement - subjective

(Physics)

eg:  $g = f * h + n$

eg:  $g = f * h + n$  (This part)

Degradation (from an accurate noise statistics)

Stochastic

(Poisson, noise)

→ Given  $g$ , how to find  $f$ ?

• We can get estimate of  $f$ ?

→ getting paired data is difficult (like getting image in same location, some lighting etc. is difficult)

## Image deblurring

Point deblurring →  $f, h$  unknown

Non-blind deblurring →  $h$  is known

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

eg: given  $h$  (like motion blur kernel)

## Well-posedness

### Hadamard's cond's for well-posedness

(Existence)

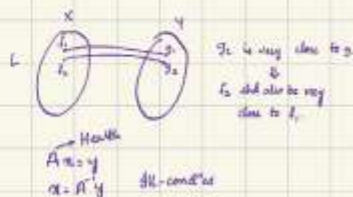
For each datum  $g$  in a given class of  $h$ ,  $\exists$  a  $w$   $f$  in a prescribed class  $x$

2. The solution  $f$  is unique in  $x$

i.e.  $L \subset L$  1-1 & onto

$L$  is infinite

3. The dependence of  $f$  on  $g$  should be continuous [Numerical stability]



15A)

1. Existence:  $A \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = y$

$$A: \mathbb{R}^n \rightarrow \mathbb{R}^m$$

$y$  is not in the span of the columns of the least squares:

$$\min_{\vec{x}} \|A\vec{x} - y\|^2$$

$$\text{Let } J(\vec{x}) = (A\vec{x} - y)^T (A\vec{x} - y)$$

$$= (\vec{x}^T A^T - y^T) (A\vec{x} - y) = \vec{x}^T A^T A \vec{x} - \vec{x}^T A^T y - y^T A \vec{x} + y^T y$$

$$= \vec{x}^T A^T A \vec{x} - 2y^T A \vec{x} + y^T y$$

$$\frac{dJ(\vec{x})}{d\vec{x}} = 2A^T A \vec{x} - 2A^T y = 0$$

$$A^T A \vec{x} = A^T y \Rightarrow \boxed{\vec{x} = (A^T A)^{-1} A^T y}$$

→  $Az \neq y$   $\because y$  is not in  $\mathcal{R}(A)$

$(A^T A)^{-1}$  is the left inverse of  $A$

$$A \hat{z} = A(A^T A)^{-1} A^T y$$

$P = A(A^T A)^{-1} A^T$

$$P^T = (A(A^T A)^{-1} A^T)^T = A(A^T A)^{-1} A^T = P$$

$$P^2 = P^T P = P A(A^T A)^{-1} A^T P = P$$

$\therefore P$  is projection matrix

$P$  is linear & projection matrix



→ What can we say abt  $(y - \hat{y})$ ?

$$(y - \hat{y})^T \hat{y} = 0$$

$$(y - \hat{y})^T \hat{y} = 0 \Rightarrow y^T \hat{y} - \hat{y}^T P y = 0$$

$$y^T P y - y^T P^T P y = 0$$

$$\text{If } P^T = P$$

$$\text{then } P^T P = P^2 = P$$

$\therefore P$  is orthogonal projection matrix [min. error w.r.t.  $\hat{z} = y$ ]

→ Here the basic assumption was  $A$  is full rank ( $n \times n$ ).

Uniqueness:  $Az = y$



If there are 2 or more vectors  $z_1, z_2$

then pick the  $z$  with least energy

min  $\|z\|^2$  st  $Az = y$  (Assumes  $z$  is real)

$$J(z) = z^T z + \frac{1}{2} \|Az - y\|^2$$

$$\frac{dJ(z)}{dz} = z + A^T(Az - y) = 0$$

$$2z = -A^T y$$

$$\frac{dJ}{dz} = Az - y = 0$$

→ When  $A$  is not full rank

$$Az = y \rightarrow \text{Pseudo inverse}$$

$$\hat{z} = A^+ y$$

$$\text{SVD: } A = U \Sigma V^T$$

$$A^+ = V \Sigma^+ U^T$$

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$$\Sigma^+ = \begin{bmatrix} \frac{1}{\sigma_1} & 0 & 0 \\ 0 & \frac{1}{\sigma_2} & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$\rightarrow \sigma_3 = 0$   $\therefore A$  is not full rank

$$\Sigma^+(U, V) = \begin{cases} \frac{1}{\sigma_i} u_i v_i^T, & \text{if } \sigma_i \neq 0 \\ 0, & \text{otherwise} \end{cases}$$

7.4

3) Continuity:

→ Matrix of  $A$  (you'd like to be invertible)

$$Az = y$$

$$z = A^{-1} y$$

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

is not invertible

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}$$

is invertible

but it is ill-conditioned

Instead we can use pseudoinverse

Normed linear operator: A linear operator  $T$  is said to be bounded if  $\exists$  a real no.  $c > 0$  st  $\forall x \in D(T)$

$$\|Tx\| \leq c\|x\|$$

$$c \geq \frac{\|Tx\|}{\|x\|} \quad x \in D(T) \setminus \{0\}$$

$\rightarrow$  The smallest possible  $c$  is the supremum of the R.N.s of  $D(T)$ . The smallest value is called the norm of  $T$  ( $\|T\|$ )

$\rightarrow \|T\| = \sup_{\substack{x \in D(T) \\ \|x\|=1}} \|Tx\|$  [norm = distance from the set  
norm  $\|x\| = 1 \rightarrow$  unit sphere  
sup  $\|Tx\| = 2$

$\rightarrow$  Consider a matrix  $A$ , its pth norm is given by  $\|A\|_p = \sup_{\substack{x \in \mathbb{R}^n \\ \|x\|_p=1}} \|Ax\|_p$

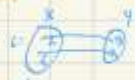
$\rightarrow$  Spectral norm  $\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)}$   
(eigen values)

$\rightarrow \|A\|_2 = \sqrt{\lambda_{\max}(A^T A)}$

Matrix inequality result:  $\lambda_{\min}(A^T A) \leq \frac{x^T A^T A x}{x^T x} \leq \lambda_{\max}(A^T A)$   
 $\Rightarrow \|A\|_2 = \sqrt{\lambda_{\max}(A^T A)} = \sqrt{\text{eigen}(A^T A)}$

$\rightarrow$  There is a sign vector  $z$  s.t.  $\|Az\|_2 = \|A\|_2$

$z = \frac{1}{\|x\|_2} A^T x$ ,  $z^T A^T A z = \lambda_{\max}(A^T A)$



If  $y = Ax$ , no claim  
we expect  $\|y\|$  to be close  
determined by "health" of  $A$

Relative Change:

$Ax = y \Rightarrow x = A^{-1}y$   
if  $y$  is perturbed  $\rightarrow$  taking "x" norm

$\|y\| \leq \|A\| \|x\| \leq \|A\| \|x\|$  [Any def. of norm]

$\frac{1}{\|x\|} \leq \|A\| \frac{1}{\|y\|} \quad \text{--- (1)}$

$\rightarrow$  Suppose  $y$  is perturbed by  $\Delta y$   
 $\rightarrow x$  gets perturbed by  $\Delta x$

$x + \Delta x = A^{-1}(y + \Delta y)$

$\Delta x = A^{-1} \Delta y$

$\| \Delta x \| \leq \|A^{-1}\| \| \Delta y \| \quad \text{--- (2)}$

$\text{--- (1) \& (2)} \Rightarrow \frac{\| \Delta x \|}{\| x \|} \leq \|A^{-1}\| \frac{\| \Delta y \|}{\| y \|}$

$\|A\| \|A^{-1}\| \geq \|AA^{-1}\| = \|I\| = 1$

$\Rightarrow \boxed{\kappa(A) = \frac{\|A\|}{\|A^{-1}\|} \geq 1}$

cond<sup>n</sup> number  $\rightarrow$  rank deficiency

$\frac{\| \Delta x \|}{\| x \|} \leq \kappa(A) \frac{\| \Delta y \|}{\| y \|}$

$\rightarrow$  scaling matrices have  $\kappa(A) = 1$

If  $\kappa(A)$  is 2, rel. change in  $x$

is twice compared to higher  $\kappa(A)$ .

Let  $y = \begin{bmatrix} 1 \\ 1.0001 \end{bmatrix}$ ,  $Ay = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$   
 $\Delta y = \begin{bmatrix} 0 \\ 0.0001 \end{bmatrix}$   
 $\Delta x = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$   
 $\| \Delta x \| = 1$   
 $\| x \| = \sqrt{2}$   
 $\| \Delta y \| = 0.0001$   
 $\| y \| = \sqrt{2}$   
 $\kappa(A) = \frac{1}{\sqrt{2}}$

$\frac{\| \Delta x \|}{\| x \|} = \kappa(A) \frac{\| \Delta y \|}{\| y \|}$   
 $\frac{1}{\sqrt{2}} = \kappa(A) \frac{0.0001}{\sqrt{2}}$   
 $\kappa(A) = 10^4$

$\Rightarrow \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}$

well have a  
high cond<sup>n</sup> no.  
 $\rightarrow$  ill conditional  
matrix

$A = A^T$   
 $\lambda_1 \approx 10^{-4}$ ,  $\lambda_2 \approx 2$

$\approx 10^4 \sqrt{\frac{\max}{\min}} = 10^4 \sqrt{\frac{2}{10^{-4}}} \approx 10^4 \sqrt{2 \times 10^4}$   
 $\rightarrow$  numerically unstable

Let  $y = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ ,  $x = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$

$\frac{1.0001 + 1.0001}{1.0001 + 1.0001} = 1$   
 $\frac{1.0001 + 1.0001}{1.0001 + 1.0001} = 1$

$x = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$



## Image deblurring:

→ SIV blur, PSF known, Noise statistics known

→  $g = h * f + n$  → deterministic

$$\hat{f} = B \hat{g} + 0$$

$$\Rightarrow \hat{f}(k, \omega) = F(k, \omega) H(k, \omega) + N(k, \omega)$$



Suppose noise is very small.



$$\hat{F}(k, \omega) = \frac{G(k, \omega)}{H(k, \omega)} = \frac{F(k, \omega) + N(k, \omega)}{H(k, \omega)}$$

→  $N(k, \omega)$  gets amplified by  $\frac{1}{H(k, \omega)}$



## Pseudo-inverse fitting:

We take modified  $H(k, \omega)$  instead of  $H(k, \omega)$

$$H_m(k, \omega) = \begin{cases} H(k, \omega), & |H(k, \omega)| > \epsilon \\ \epsilon, & \text{o.w.} \end{cases}$$

$$\Rightarrow \hat{F}(k, \omega) = \frac{F(k, \omega) + N(k, \omega)}{H_m(k, \omega)} \rightarrow \text{blurred image with less noise}$$

Regularization → by  $\epsilon$

Trade off b/w bluriness & noise

→ Suppose we also had prior knowledge abt  $f$   
 ↳ (generic prior - eg. face, being specific)

→ Cost fn

$$J(x) = \|y - Ax\|^2 + \lambda \|Qx\|^2 \rightarrow \text{regularization const.}$$

Tikhonov - Matrix Regularization

$\lambda$  → regularizing factor

can be identity  $Q = I$  ( $\|x\|^2$  → norm of  $x$  shd be small)

$Q$  Laplacian (if we know  $f$  is a smooth image)



$$\rightarrow J(x) = (y - Ax)^T (y - Ax) + \lambda (Qx)^T (Qx)$$

$$= (y^T - x^T A^T) (y - Ax) + \lambda (x^T Q^T) (Qx)$$

$$= y^T y - y^T A x + x^T A^T A x + \lambda x^T Q^T Q x$$

$$\frac{dJ(x)}{dx} = 0 = -2A^T y + 2A^T A x + 2\lambda Q^T Q x = 0$$

$$\Rightarrow A^T A x + \lambda Q^T Q x = A^T y$$

$$\Rightarrow \boxed{\frac{1}{2} (A^T A + \lambda Q^T Q)^{-1} A^T y}$$

→ Instead of inverting  $A^T A$  like you hope  
 we are stabilizing  $A$  by adding  $\lambda Q^T Q$ .

→ The theory of regularization is a formal process of incorporating prior knowledge to robustly estimate the latent unknown (clean blurred image).

# Stochastic Regularization (Nature of MSE)

→ MSE of a RV 'Y' by a const 'c'

MSE is given by  $E[(Y-c)^2] = \int (y-c)^2 f(y) dy$  &c

For 'c' to be min

$$\frac{dc}{dc} = \int (y-c) f(y) dy = 0$$

$$\int y f(y) dy = \int c f(y) dy \Rightarrow \boxed{c = E[Y]}$$

0.1.01

**Non-linear MSE:** Estimation not by a const but by a fcn c(x) of the RV X (assuming X carries info abt Y)

Find:

$$E[(Y-c(x))^2]$$

Find c(x) st 'E' is min

$$E[(Y-c(x))^2], E[(Y-g(x))^2]$$

↳ Linear non-linear fcn of X

Law of iterated expectation:

$$\rightarrow E[E[Y|X]] = E[Y]$$

$$\rightarrow c: E[(Y-c(x))^2] = \iint (y-c(x))^2 f_{Y,X}(y,x) dy dx = \iint (y-c(x))^2 f_{Y|X}(y|x) f_X(x) dy dx$$

$$= \int f_X(x) \left[ \int (y-c(x))^2 f_{Y|X}(y|x) dy \right] dx$$

$$\text{shd be min w.r.t c(x)} \quad \bar{c} = \int (y-c(x))^2 f_{Y|X}(y|x) dy$$

$$\frac{dc}{dc(x)} = \int -2(y-c(x)) f_{Y|X}(y|x) dy = 0$$

$$\Rightarrow \int y f_{Y|X}(y|x) dy = \int c(x) f_{Y|X}(y|x) dy = c(x)$$

$$\Rightarrow \boxed{c(x) = E[Y|x]}$$

↳ Usually non-linear fcn of x

$$\text{Proof: } E[E[Y|X]] = \int \phi(x) f_X(x) dx$$

$$= \int \left( \int y f_{Y|X}(y|x) dy \right) f_X(x) dx$$

$$= \int \int y f_{Y|X}(y|x) f_X(x) dy dx$$

$$= \int y \left( \int f_{Y|X}(y|x) f_X(x) dx \right) dy = \int y f_Y(y) dy = E[Y]$$

→ If we constrain c(x) to be linear, we get suboptimal result.

**Linear MSE:** Willing to settle down for a linear estimation although it will be suboptimal.

The linear MSE of Y given RV's  $X_i$   $i=1,2,\dots,n$  is given by

$$\hat{Y} = \sum_{i=1}^n a_i X_i$$

Find  $a_i$ 's st  $E[(Y-\hat{Y})^2]$  is min.

$$E[(Y-a_1 X_1 - a_2 X_2 - \dots - a_n X_n)^2]$$

$$\frac{dc}{da_i} = 0 \Rightarrow E[2(Y-a_1 X_1 - \dots - a_n X_n)(-X_i)] = 0 \quad \forall i: 1 \text{ to } n$$

Orthogonality principle for linear MSE

$$\Rightarrow E[Y X_i - a_1 X_1 X_i - a_2 X_2 X_i - \dots - a_n X_n X_i] = 0$$

$$E[a_1 X_1 X_i + a_2 X_2 X_i + \dots + a_n X_n X_i] = E[Y X_i]$$

$$\text{Let } r_{Yi} = E[Y X_i]$$

$$r_{ii} = E[X_i X_i]$$

$$R_B = I \Rightarrow \boxed{a = R^{-1} I}$$

$$\Rightarrow \begin{bmatrix} r_{11} & r_{12} & \dots & r_{1n} \\ r_{21} & r_{22} & \dots & r_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ r_{n1} & r_{n2} & \dots & r_{nn} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} r_{Y1} \\ r_{Y2} \\ \vdots \\ r_{Yn} \end{bmatrix}$$

\* Find LMSE of Y given  $X$ :  $Y = aX + b$ . Find  $a$  &  $b$ .

$$c = E[(Y-aX-b)^2]$$

$$\frac{dc}{da} = 0 \Rightarrow E[2(Y-aX-b)(-X)] = 0$$

$$E[(Y-aX-b)X] = 0$$

$$\frac{dc}{db} = 0 \Rightarrow E[2(Y-aX-b)(-1)] = 0$$

$$E[(Y-aX-b)] = 0$$

General Orthogonality principle: For non-linear estimation.

If  $g(x)$  is a non-linear MSE of  $y$  i.e.  $g(x) = E[y|x]$  then the estimation error  $(y - g(x))$  is orthogonal to any fn  $w(x)$  [linear or non-linear]

Pr: Let  $g(x) = E[y|x]$

TPT  $E[(y - g(x)) w(x)] = 0$

example  $E[g(x) w(x)] = E[w(x) g(x)]$

$$= E[E[w(x) y | x]] = \int f_x(x) \int w(x) y f_{y|x}(y|x) dy dx$$

$$= \iint w(x) y f_{y|x}(y|x) f_x(x) dy dx$$

$$= E[w(x) y]$$



Normality Theorem If RVs  $y, x_1, \dots, x_n$  are jointly normal (Gaussian) with zero mean, then the LMSE & NLMS of  $y$  are identical

Pr: Let  $\hat{y} = \sum_{i=1}^n a_i x_i$  be a linear estimator of  $y$ .

Claim:  $(y - \hat{y})$  &  $x_i$  are jointly Gaussian.

$$\begin{bmatrix} y - \hat{y} \\ x_1 \\ \vdots \\ x_n \end{bmatrix} \rightarrow \begin{bmatrix} 1 - a_1 - \dots - a_n & -a_1 & \dots & -a_n \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix} \begin{bmatrix} y \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$$

$\downarrow$  linear function of  $y, x_1, \dots, x_n$        $\downarrow$  jointly Gaussian       $\downarrow$  jointly Gaussian

Claim: Zero mean orthogonal RVs are uncorrelated.

$$E[(x_i - y\hat{y})(y - y\hat{y})] = E[x_i y] = 0$$

$\rightarrow$  By the orthogonality principle of LMSE

$$E[(y - \hat{y}) x_i] = 0$$

$\rightarrow$  Because  $(y - \hat{y})$  &  $x_i$  are jointly Gaussian & uncorrelated  $\Rightarrow y - \hat{y} \perp x_i$  are indep.

$$y - \hat{y} \perp x_i$$

$$\rightarrow E[y - \hat{y} | x] = E[y - \hat{y}] = 0$$

$$E[y|x] - \underbrace{E[\hat{y}|x]}_{\hat{y}} = 0 \Rightarrow \hat{y} = E[y|x]$$

$\downarrow$  Non-linear MSE

Wiener Filter:

Wiener Filter: (Linear)

$\rightarrow$  Find an estimate of the original (clean) image  $f(m,n)$  given a blurred observation  $g(m,n)$  of the original image

$\rightarrow \hat{f}(m,n) = E[f(m,n) | g(m,n)]$  :  $1 \leq m, n \leq N$   $\rightarrow$  But NLMS [Non-linear in  $g$  usually]

$\rightarrow$  We settle for a linear <sup>(optimal)</sup> estimate [suboptimal]

$$\hat{f} = \hat{H} g$$

$\downarrow$  matrix of constants

$\rightarrow$  Find the optimal  $\hat{H}$  can be done using the orthogonality principle

$$E[(f - \hat{f}) g^T] = 0 \rightarrow \text{Matrix of zeros}$$

$$E[f g^T] - \hat{H} E[g g^T] = 0 \Rightarrow E[f g^T] = E[\hat{f} g g^T]$$

$$= \hat{H} E[g g^T]$$

Blurring op.  $g = H f + n$  <sup>zero mean, i.i.d</sup> [img formation model]

$$g g^T = (H f + n)(H f + n)^T = H f f^T H^T + H f n^T + n f^T H^T + n n^T$$

$$f g^T = f f^T H^T + f n^T$$



$$\Rightarrow E\{FF^T H^T + F n^T\} = \hat{H} E\{H F F^T H^T + H F n^T + n^T H^T + n n^T\}$$

$$E\{FF^T\} H^T + E\{F n^T\} = \hat{H} [H E\{FF^T\} H^T + H E\{F n^T\} + E\{n n^T\} H^T + E\{n n^T\}]$$

$$E\{FF^T\} H^T = \hat{H} [H E\{FF^T\} H^T + E\{n n^T\}]$$

$$R_F H^T = \hat{H} [H R_F H^T + R_n]$$

$$\hat{H} = R_F H^T [H R_F H^T + R_n]^{-1}$$

↓  
Post reg. matrix  
Pre reg. matrix

$$\Rightarrow (\hat{A} + B C D)^{-1} = A^{-1} - A^{-1} B (C^{-1} + D A^{-1} B)^{-1} D A^{-1}$$

$$\Rightarrow \hat{H} = (H^T R_F^{-1} H + R_F^{-1})^{-1} H^T R_F^{-1}$$

Wiener Filter: Frequency interpretation of WF

$\hat{h} \rightarrow$  space-invariant

$R_F, R_n \rightarrow$  DBC matrices

$$\Rightarrow \hat{z} = \hat{H} g = R_F H^T (H R_F H^T + R_n)^{-1} g$$

$$\text{DFT} \quad \hat{F} \hat{L} = \hat{F} R_F H^T (H R_F H^T + R_n)^{-1} \hat{F}^* \hat{G}$$

$$\hat{F} = \hat{F} R_F \hat{F}^* \hat{F} L^T (H R_F H^T + R_n)^{-1} \hat{F}^* \hat{G}$$

$$(\hat{F} (H R_F H^T + R_n)^{-1} \hat{F}^*)^{-1}$$

$$(\hat{F} H R_F H^T \hat{F}^* + \hat{F} R_n \hat{F}^*)^{-1}$$

$$\hat{F} R_F \hat{F}^* \rightarrow \text{FSD of } F$$

is in  
frequency domain

$$\Rightarrow \hat{F} = S_{FF} H^* (H S_{FF} H^* + S_{nn})^{-1} \hat{G}$$

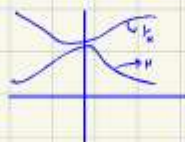
$$\Rightarrow \hat{F}(k, \omega) = \frac{S_{FF}(k, \omega) \cdot H^*(k, \omega) \cdot G(k, \omega)}{H(k, \omega) S_{FF}(k, \omega) H^*(k, \omega) + S_{nn}(k, \omega)}$$

$$= \frac{S_{FF}(k, \omega) H^*(k, \omega)}{|H(k, \omega)|^2 S_{FF}(k, \omega) + S_{nn}(k, \omega)} G(k, \omega)$$

$$\hat{F}(k, \omega) = \frac{H^*(k, \omega)}{|H(k, \omega)|^2 \frac{S_{nn}(k, \omega)}{S_{FF}(k, \omega)} + 1} G(k, \omega)$$

Special case:

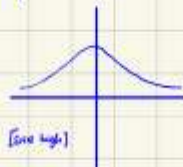
a)  $\frac{1}{T} \quad n=0 \quad \hat{H}(k, \omega) = \frac{1}{H(k, \omega)} \quad (\text{Wiener filter})$



b) No Noise  $H(k, \omega) = 1 \quad \forall \omega$

$$\Rightarrow \hat{H}(k, \omega) = \frac{1}{1 + \frac{S_{nn}(k, \omega)}{S_{FF}(k, \omega)}} = \frac{SNR}{1 + SNR}$$

$\begin{cases} 1 & \text{Low frequency (low noise)} \\ \frac{1}{1 + SNR} & \text{at high } \omega \end{cases}$



c) Blue + noise



2.1.4

$$\rightarrow F_{xy}(k) \in [x(n) x(n-k)]$$

$$= \sum_{n=-\infty}^{\infty} \sum_{k=-\infty}^{\infty} x(n) x(n-k) e^{-j\omega k}$$

$$= \sum_{k=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} x(n) x(n-k) e^{-j\omega k} = \underbrace{\sum_{k=-\infty}^{\infty} x(k) e^{-j\omega k}}_{X(e^{j\omega})} \underbrace{\sum_{n=-\infty}^{\infty} x(n) e^{-j\omega n}}_{X(e^{j\omega})}$$

$$= X^*(e^{j\omega}) X(e^{j\omega})$$

$$\Rightarrow \boxed{P_{xx}(\omega) = |X(e^{j\omega})|^2}$$

$$\rightarrow \text{solve filter: } \hat{P}(k, \theta) = \frac{H^*(k, \theta)}{|H(k, \theta)|^2 + \frac{\lambda_1 + \lambda_2}{S_y(k, \theta)}}$$

How to choose the priors?

If we know the blurred image is a face, we can get it from previously taken average of clear images.

$$= \frac{1}{M} \sum_{m=1}^M |x_m(e^{j\omega})|^2$$